Data Mining

Week 6

Submitter - Himanshu Singh

Fraud Waste Abuse Detection in Healthcare Insurance - Project Milestone 1

Data Selection and EDA

Problem Statement

The primary objective of this project is to develop a predictive model to identify Fraud, Waste, and Abuse (FWA) among healthcare providers. By analyzing claims data, the project aims to "Predict potentially fraudulent providers" while uncovering the most significant variables that signal suspicious behavior. This intelligence allows organizations to flag high-risk claims for in-depth investigation, ensuring that resources are directed toward the most likely instances of financial leakage.

Problem Description

Healthcare insurance is a vital pillar of social stability, protecting citizens from the financial hardships of medical uncertainty. However, the integrity of both public and private insurance schemes is constantly threatened by Fraud, Waste, and Abuse (FWA):

Fraud: Intentional deception or misrepresentation, such as billing for services never rendered or "phantom" patients. Waste: Overutilization of services or practices that result in unnecessary costs, such as performing excessive diagnostic tests that are not medically required. Abuse: Actions that are inconsistent with sound fiscal, business, or medical practices, resulting in improper payment (e.g., upcoding or unbundling procedures).

FWA is often an organized crime involving collusion between providers, physicians, and beneficiaries. Specifically, Provider Fraud is a massive burden on Medicare; for instance, providers may exaggerate billing amounts or facilitate benefits for unauthorized individuals.

The cost of misidentification is high: failing to detect FWA leads to substantial financial losses, while incorrectly flagging legitimate providers (False Positives) causes administrative friction and delays essential patient care. A precise, data-driven approach is required to balance aggressive cost recovery with the smooth adjudication of valid claims.

Real-Life Examples in FWA Detection

Fraud (Intentional Deception): A provider submits claims for a series of complex MRIs for patients who, according to demographic data, live 500 miles away and never visited the clinic. Data Mining Perspective: We can look for geospatial anomalies and a lack of corresponding primary care referrals.

Waste (Unnecessary Costs): A clinic consistently orders expensive "comprehensive metabolic panels" for every single patient visit, regardless of the diagnosis (e.g., even for a simple flu shot). Data Mining Perspective: This is identified through frequency analysis and comparing a provider's behavior against the "peer group" average for the same specialty.

Abuse (Improper Practice): A surgeon performs a single operation but bills for three separate "sub-procedures" to increase the payout (known as unbundling). Data Mining Perspective: We can use Association Rule Mining to find procedure codes that should logically be billed under a single global fee but are appearing as separate line items.

Source of data: Elsevier''s Mendeley Data repository is a participating member of the National Institutes of Health (NIH) Office of Data Science Strategy (ODSS) GREI project. liang, tim (2024), "HEALTHCARE PROVIDER FRAUD DETECTION ANALYSIS", Mendeley Data, V1, doi: 10.17632/gsn2hyty37.1 https://data.mendeley.com/datasets/gsn2hyty37/1

In [286…

```
import pandas as pd
import os
import numpy as np
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
from tqdm import tqdm
```

In [287…

```
data = pd.read_csv('Train.csv')
data_beneficiary = pd.read_csv('Train_Beneficiarydata.csv')
data_inpatient = pd.read_csv('Train_Inpatientdata.csv')
data_outpatient = pd.read_csv('Train_Outpatientdata.csv')

print("data : ",data.shape)
print("beneficiary_data : ",data_beneficiary.shape)
print("inpatient_data : ",data_inpatient.shape)
print("outpatient_data : ",data_outpatient.shape)
```

```
data :  (5410, 2)
beneficiary_data :  (138556, 25)
inpatient_data :  (40474, 30)
outpatient_data :  (517737, 27)
```

Exploratory Data Analysis

1 . Procedure Code and Diagnosis Codes Analysis

Inpatient Records Data Analysis

Inpatient Data - Inpatient Data refers to records for patients who have been formally admitted to a hospital or medical facility for at least one night.

In [288…

```
#Getting all the columns of Inpatient data
print('Columns in this dataset are : ')
print(data_inpatient.columns)

data_inpatient.head(5)
```

```
Columns in this dataset are :
Index(['BeneID', 'ClaimID', 'ClaimStartDt', 'ClaimEndDt', 'Provider',
       'InscClaimAmtReimbursed', 'AttendingPhysician', 'OperatingPhysician',
       'OtherPhysician', 'AdmissionDt', 'ClmAdmitDiagnosisCode',
       'DeductibleAmtPaid', 'DischargeDt', 'DiagnosisGroupCode',
       'ClmDiagnosisCode_1', 'ClmDiagnosisCode_2', 'ClmDiagnosisCode_3',
       'ClmDiagnosisCode_4', 'ClmDiagnosisCode_5', 'ClmDiagnosisCode_6',
       'ClmDiagnosisCode_7', 'ClmDiagnosisCode_8', 'ClmDiagnosisCode_9',
       'ClmDiagnosisCode_10', 'ClmProcedureCode_1', 'ClmProcedureCode_2',
       'ClmProcedureCode_3', 'ClmProcedureCode_4', 'ClmProcedureCode_5',
       'ClmProcedureCode_6'],
      dtype='object')
```

Out[288…

	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	Provider	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician	OtherPhysician	AdmissionDt	…	ClmDiagnosisCode_7	ClmDiagno
0	BENE11001	CLM46614	2009-04-12	2009-04-18	PRV55912	26000	PHY390922	NaN	NaN	2009-04-12	…	2724	
1	BENE11001	CLM66048	2009-08-31	2009-09-02	PRV55907	5000	PHY318495	PHY318495	NaN	2009-08-31	…	NaN	
2	BENE11001	CLM68358	2009-09-17	2009-09-20	PRV56046	5000	PHY372395	NaN	PHY324689	2009-09-17	…	NaN	
3	BENE11011	CLM38412	2009-02-14	2009-02-22	PRV52405	5000	PHY369659	PHY392961	PHY349768	2009-02-14	…	25062	
4	BENE11014	CLM63689	2009-08-13	2009-08-30	PRV56614	10000	PHY379376	PHY398258	NaN	2009-08-13	…	5119	

5 rows × 30 columns

We will try to analyze the data we have

This data contains Diagnosis Code and Procedure Codes

- Diagnosis Codes (primarily ICD-10 codes) are the standardized alphanumeric strings used to describe a patient's medical condition, symptoms, or injuries.
- Procedure Codes are standardized numeric or alphanumeric codes used to identify the specific medical, surgical, or diagnostic services provided to a patient. While Diagnosis Codes explain "why" the patient is there, Procedure Codes explain "what" was done to them.

Since there are multiple service and procedure code associated to each claim we will combine them together and sort, so that it doesn't give us multiple sets of same procedure and services

Eg : Knee Surgery will have Anaesthesia + Arthroscopically aided ACL reconstruction, the sort and combine will give us combination of procedure.

We will try to find which Procedure and Diagnonsis are the top reported in the dataset we got.

In [289…

```
# Function to analyse categorical columns

def analyse_cat_columns(dataset, col_to_analyse, prefix='', title='Distribution in percentage',
                        top_val=30, y_lim=None, color='green'):
    """Plots a bar graph for the percentage distribution of a categorical column."""

    # Calculate counts and convert to percentage in one step
    counts = dataset[col_to_analyse].value_counts().head(top_val)
    percentages = (counts / len(dataset)) * 100

    # Create the plot labels with the optional prefix
    labels = [f"{prefix}{label}" for label in percentages.index]

    # Plotting
    plt.figure(figsize=(15, 5))
```

```
    plt.bar(labels, percentages.values, color=color)

    # Formatting
    plt.title(title)
    plt.ylabel('Percentage (%)')
    plt.xlabel(col_to_analyse)
    plt.xticks(rotation=45)
    plt.grid(axis='y', linestyle='--', alpha=0.7)

    if y_lim is not None:
        plt.yticks(y_lim)

    plt.tight_layout()
    plt.show()
```

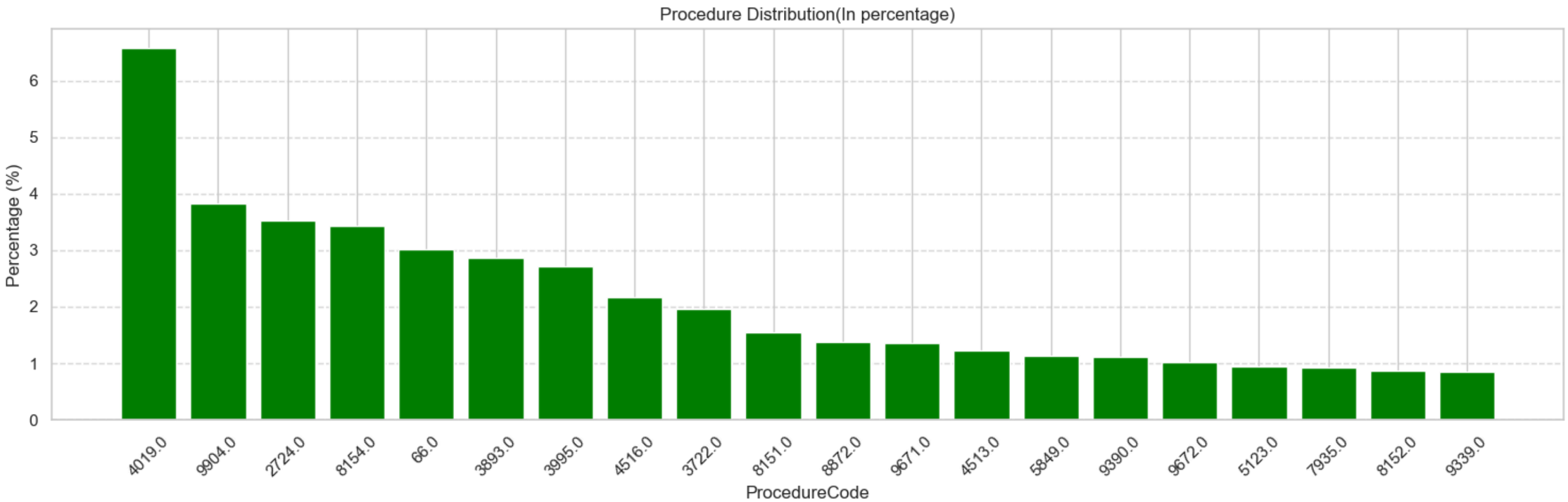
In [290…

```
# Select all columns that start with 'ClmProcedureCode'
proc_cols = [col for col in data_inpatient.columns if 'ClmProcedureCode' in col]

# Concatenate them all and drop NaNs
inpatient_df = pd.DataFrame(
    pd.concat([data_inpatient[col] for col in proc_cols], axis=0).dropna(),
    columns=['ProcedureCode']
)

print(f"Total Procedures Found: {inpatient_df.shape[0]}")

print(inpatient_df.shape)
```

```
Total Procedures Found: 29692
(29692, 1)
```

In [291…

```
analyse_cat_columns(inpatient_df,col_to_analyse='ProcedureCode' ,title = 'Procedure Distribution(In percentage)',top_val = 20)
```



Conclusion

4019.0 (Other biopsy of lymphatic structures) is the most used procedure code in the dataset. About 6% claims have that procedure code. Since this is the most common procedure, it should be the first target for "Peer Comparison" analysis to see if specific providers are billing it at a rate significantly higher than the 6.6% average

The top 5 procedures—4019, 9904 (Blood Transfusion), 2724 (Hip Replacement/Repair), 8154 (Total Knee Replacement), and 66 (CS Section/Other)—represent the core surgical and clinical volume. FWA Impact: High-cost procedures like 8154 and 2724 are high-risk areas for "Upcoding," where a simple repair might be billed as a complex total replacement.

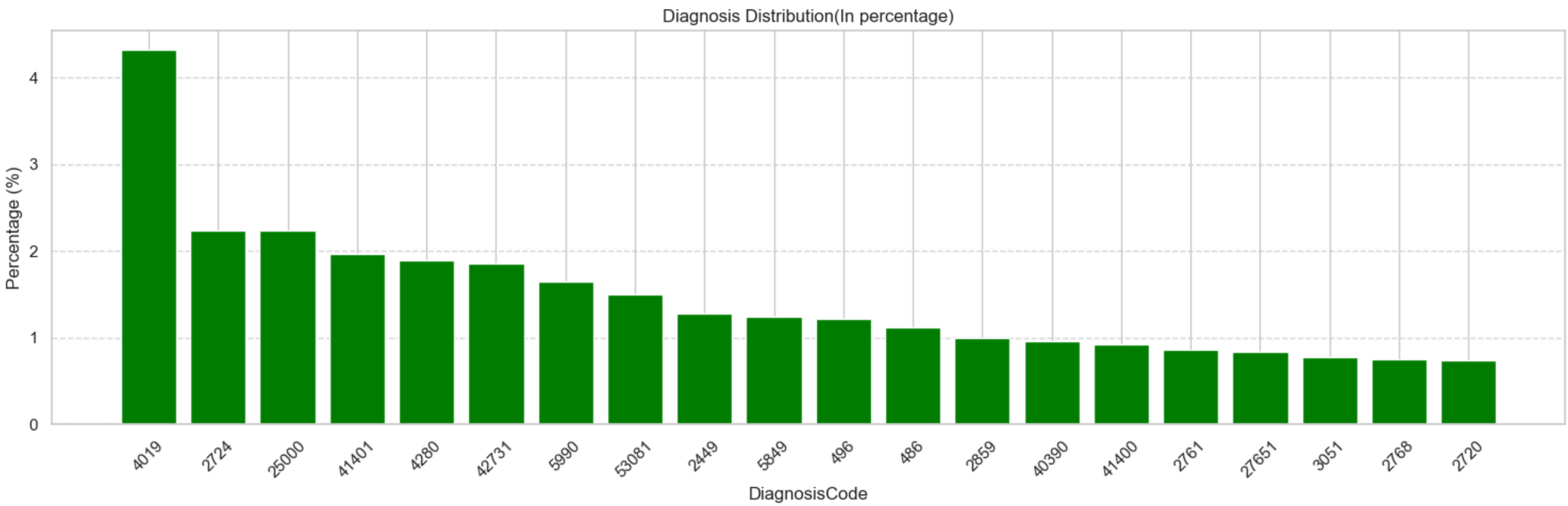
In [292…

```
# Automatically grabs ClmDiagnosisCode_1 through ClmDiagnosisCode_10
diag_cols = data_inpatient.filter(like='ClmDiagnosisCode').columns

# Concatenate into a single Series and remove missing values
inpatient_diag_df = pd.DataFrame(
    pd.concat([data_inpatient[col] for col in diag_cols], axis=0).dropna(),
    columns=['DiagnosisCode']
)

print(f"Total Diagnosis Occurrences: {inpatient_diag_df.shape[0]}")
print(inpatient_diag_df.shape)
```

```
Total Diagnosis Occurrences: 327328
(327328, 1)
```

In [293…

```
analyse_cat_columns(inpatient_diag_df,col_to_analyse='DiagnosisCode' ,
                    title = 'Diagnosis Distribution(In percentage)',
                    top_val = 20, color ='green')
```



Conclusion

Interestingly 4019 is the most used Diag code in the dataset which coincidently looks like the Procedure code 4019. About 4% claims have that diag code.

The top 5 diagnosis codes—4019 (Hypertension), 2724 (High Cholesterol), 25000 (Type 2 Diabetes), 41401 (Coronary Artery Disease), and 4280 (Congestive Heart Failure)—are all chronic conditions. FWA Insight: These are frequently used as "secondary" diagnoses to increase the complexity level of a claim, which can lead to higher reimbursement (a practice known as "Coding Creep").

Outpatient Records Data Analysis

Outpatient Data - In healthcare analytics, Outpatient Data refers to medical services provided to patients who are not admitted to a hospital (stays lasting less than 24 hours). This dataset is typically much larger and more "noisy" than inpatient data because it covers a vast range of routine interactions.

In [294…

```
#Getting all the columns of Outpatient data
print('Columns in this dataset are : ')
print(data_outpatient.columns)

data_outpatient.head(5)
```

```
Columns in this dataset are :
Index(['BeneID', 'ClaimID', 'ClaimStartDt', 'ClaimEndDt', 'Provider',
       'InscClaimAmtReimbursed', 'AttendingPhysician', 'OperatingPhysician',
       'OtherPhysician', 'ClmDiagnosisCode_1', 'ClmDiagnosisCode_2',
       'ClmDiagnosisCode_3', 'ClmDiagnosisCode_4', 'ClmDiagnosisCode_5',
       'ClmDiagnosisCode_6', 'ClmDiagnosisCode_7', 'ClmDiagnosisCode_8',
       'ClmDiagnosisCode_9', 'ClmDiagnosisCode_10', 'ClmProcedureCode_1',
       'ClmProcedureCode_2', 'ClmProcedureCode_3', 'ClmProcedureCode_4',
       'ClmProcedureCode_5', 'ClmProcedureCode_6', 'DeductibleAmtPaid',
       'ClmAdmitDiagnosisCode'],
      dtype='object')
```

Out[294…

	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	Provider	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician	OtherPhysician	ClmDiagnosisCode_1	…	ClmDiagnosisCode_9	C
0	BENE11002	CLM624349	2009-10-11	2009-10-11	PRV56011	30	PHY326117	NaN	NaN	78943	…	NaN	
1	BENE11003	CLM189947	2009-02-12	2009-02-12	PRV57610	80	PHY362868	NaN	NaN	6115	…	NaN	
2	BENE11003	CLM438021	2009-06-27	2009-06-27	PRV57595	10	PHY328821	NaN	NaN	2723	…	NaN	
3	BENE11004	CLM121801	2009-01-06	2009-01-06	PRV56011	40	PHY334319	NaN	NaN	71988	…	NaN	
4	BENE11004	CLM150998	2009-01-22	2009-01-22	PRV56011	200	PHY403831	NaN	NaN	82382	…	NaN	

5 rows × 27 columns

Same as Inpatient data, we will try to see which Diag and Procedure code are highly used in Outpatient data.

In [295…

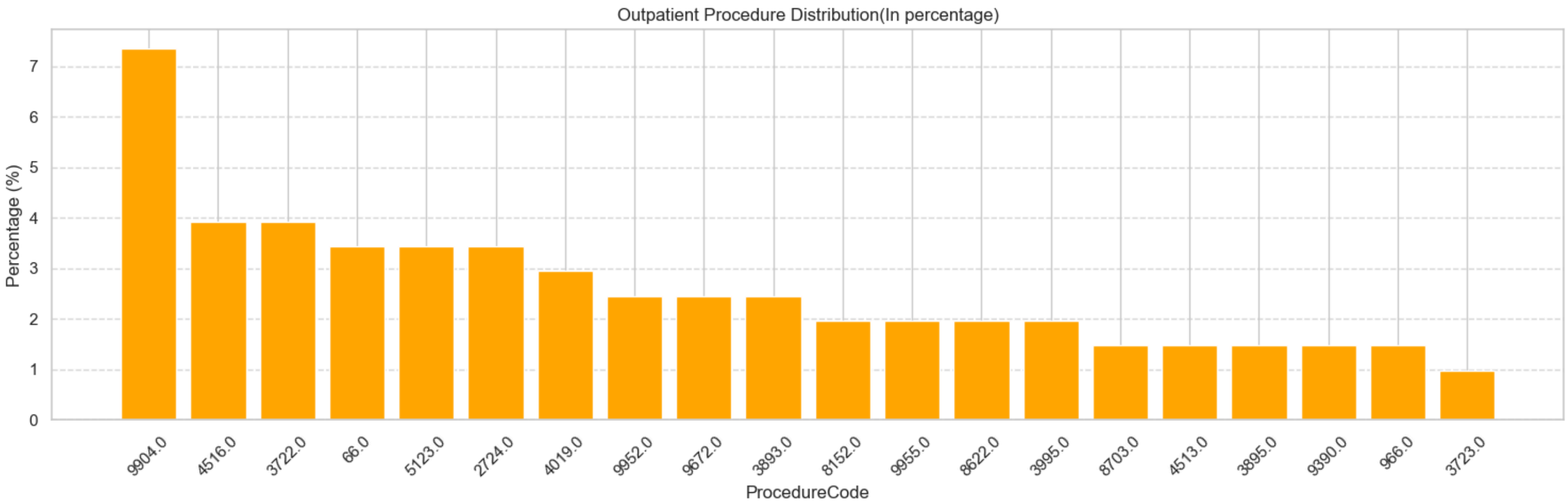
```
# Select all columns that start with 'ClmProcedureCode'
proc_cols = [col for col in data_outpatient.columns if 'ClmProcedureCode' in col]

# Concatenate them all and drop NaNs
outpatient_df = pd.DataFrame(
    pd.concat([data_outpatient[col] for col in proc_cols], axis=0).dropna(),
    columns=['ProcedureCode']
)

print(f"Total Procedures Found: {outpatient_df.shape[0]}")
```

```
print(outpatient_df.shape)
```

```
Total Procedures Found: 204
(204, 1)
```

In [296…

```
analyse_cat_columns(outpatient_df,col_to_analyse='ProcedureCode' ,title = 'Outpatient Procedure Distribution(In percentage)',top_val = 20, color='orange')
```



Conclusion

9904.0 (Packed Cell Transfusion) is the most frequent outpatient procedure, appearing in 7.5% of records. FWA Insight: High volumes of transfusions in an outpatient setting are unusual and may suggest "Infusion Suite" fraud or billing for services that should require inpatient oversight.

The presence of 3722 (Extracorporeal circulation auxiliary to open heart surgery) and 2724 (Net or tissue graft) in the top 5 suggests outpatient data contains many high-intensity "Day Surgeries."

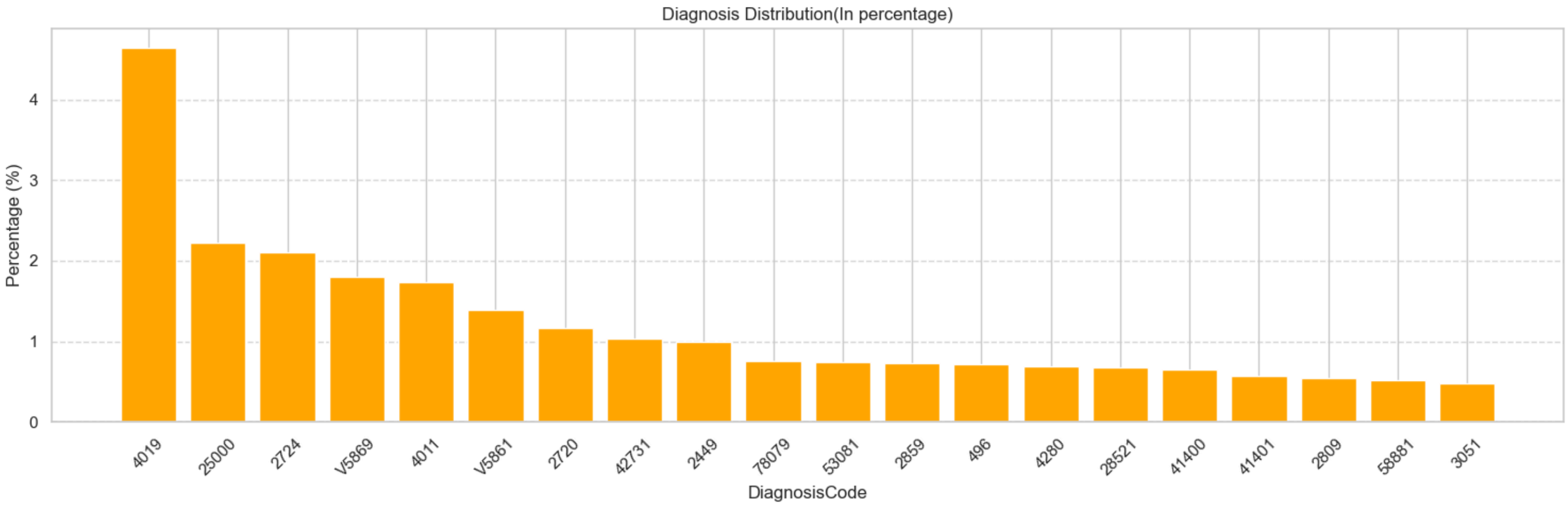
In [297…

```
# Automatically grabs ClmDiagnosisCode_1 through ClmDiagnosisCode_10
diag_cols = data_outpatient.filter(like='ClmDiagnosisCode').columns

# Concatenate into a single Series and remove missing values
outpatient_diag_df = pd.DataFrame(
    pd.concat([data_outpatient[col] for col in diag_cols], axis=0).dropna(),
    columns=['DiagnosisCode']
)

print(f"Total Diagnosis Occurrences: {outpatient_diag_df.shape[0]}")
print(outpatient_diag_df.shape)
```

```
Total Diagnosis Occurrences: 1353388
(1353388, 1)
```

In [298…

```
analyse_cat_columns(outpatient_diag_df,col_to_analyse='DiagnosisCode' ,
                    title = 'Diagnosis Distribution(In percentage)',
                    top_val = 20, color ='orange')
```



Conclusion

Diagnosis code 4019 (Unspecified Essential Hypertension) is the leading diagnosis, identified in 4.8% of all outpatient claims.

The top codes—4019, 25000 (Diabetes), and 2724 (High Cholesterol)—show that outpatient services are heavily utilized for managing long-term metabolic health.

The inclusion of V5869 (Long-term use of other medications) in the top 5 is highly significant. In outpatient data, this code is often used to justify frequent follow-up visits and lab tests. FWA Insight: Excessive use of V-codes alongside routine office visits can sometimes be a sign of "Churning," where providers schedule unnecessary appointments under the guise of "monitoring medication."

2 . Physician Analysis on number of claims

Why this is critical for Fraud, Waste, and Abuse (FWA) Identification of "Super-Billers": If one physician is filing significantly more claims than their peers, it could indicate "Phantom Billing" (billing for patients they never actually saw) or "Churning" (requiring unnecessary follow-up visits).

Capacity Analysis: In some fraud cases, data mining reveals that a single physician "performed" more surgeries in a 24-hour period than is humanly possible. This visualization helps spot those "impossible" volumes.

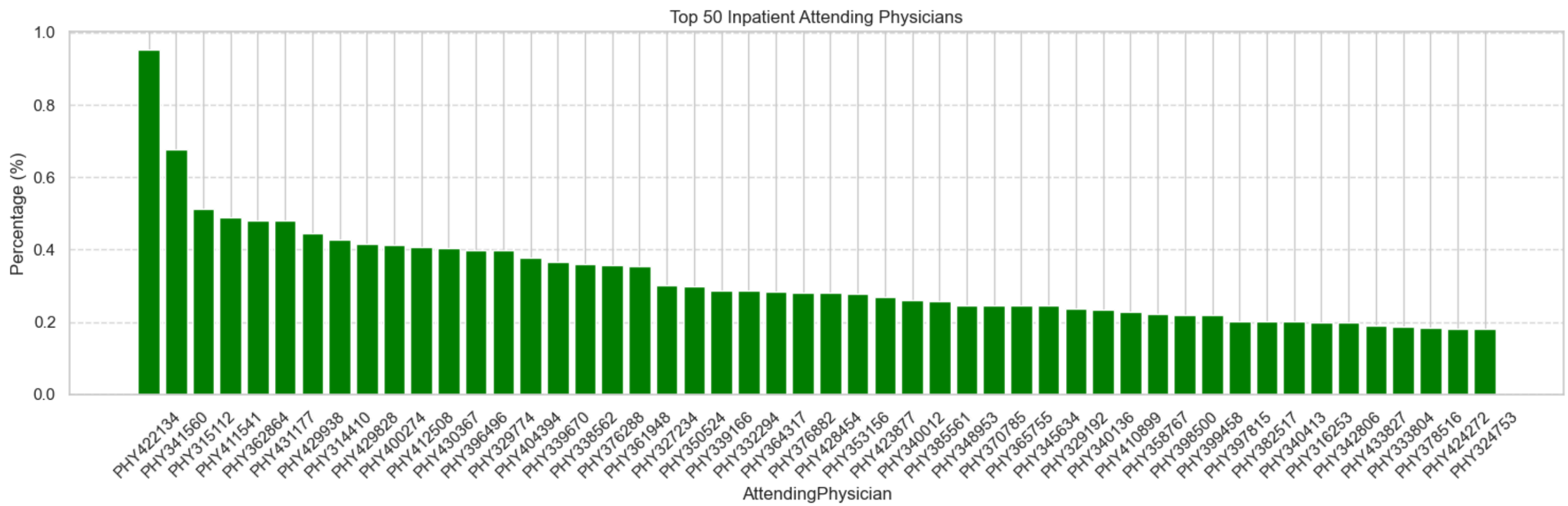
Audit Prioritization: Instead of auditing every provider, investigators use this specific list to prioritize "High-Yield" audits—focusing where the most money (and potentially the most fraud) is concentrated.

Attending Physician: The doctor who has primary responsibility for the patient's care and treatment during the entire hospital stay or clinic visit. They "own" the patient's record and oversee the overall medical plan.

Operating Physician: The specific doctor who performs a surgical or non-surgical procedure. In many outpatient cases, the Attending and Operating physician are the same person, but in complex cases, they differ.

In [299…

```
# Analyse Attending Physicians in Inpatient Data

analyse_cat_columns(data_inpatient,
                    col_to_analyse='AttendingPhysician',
                    title='Top 50 Inpatient Attending Physicians',
                    top_val=50,
                    color='green')
```



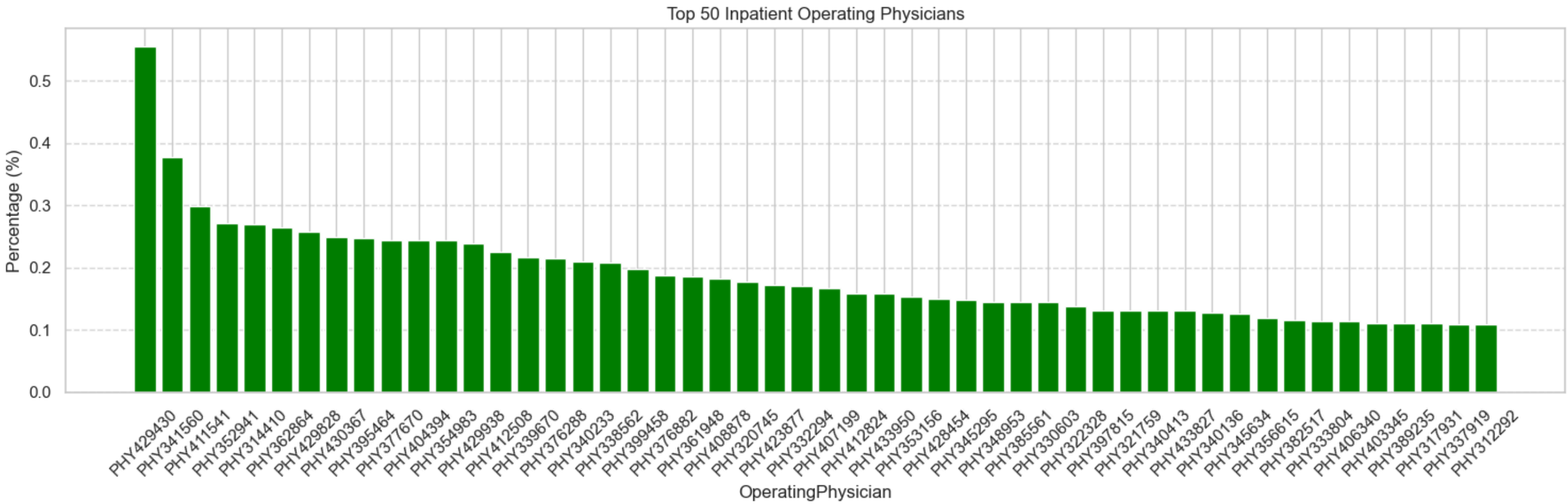
Conclusion

Analysis of the Attending Physician distribution reveals a high concentration of claims among the top 20 providers. In a fraud detection context, these 'Super-Billers' represent the highest financial risk. Further investigation is required to determine if their high volume is justified by their specialty or if it represents an anomaly such as excessive service utilization (Waste) or fictitious billing (Fraud).

Most patients are attended by physician PHY422134. Around 1% of the patients are attended by physician PHY422134

In [300…

```
# Analyse Operating Physicians in Inpatient Data

analyse_cat_columns(data_inpatient,
                    col_to_analyse='OperatingPhysician',
                    title='Top 50 Inpatient Operating Physicians',
                    top_val=50,
                    color='green')
```

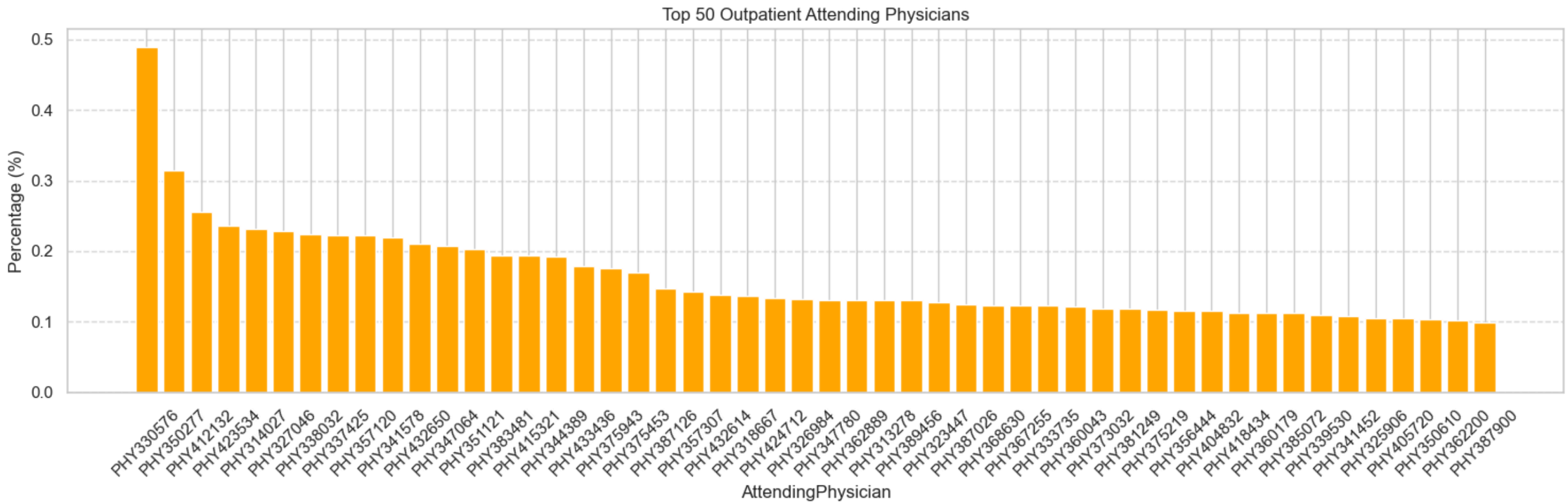


Conclusion

Most patients are operated by physician PHY429430. Around 0.55% of the patients are operated by physician PHY429430

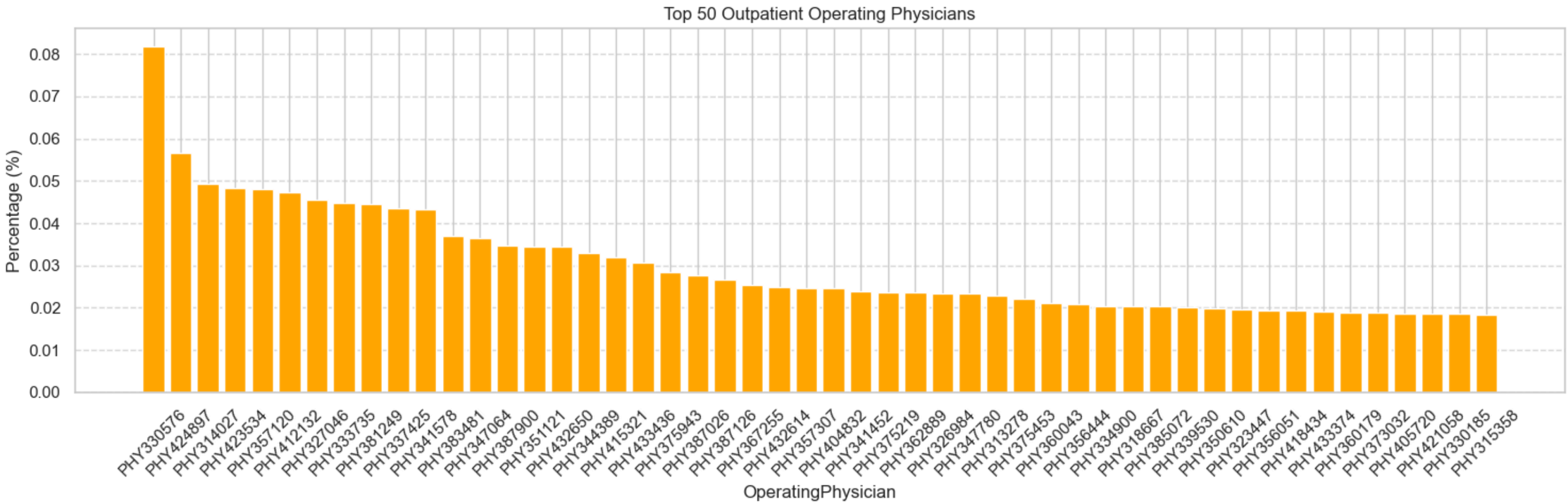
In [301…

```
# Analyse Attending Physicians in Outpatient Data

analyse_cat_columns(data_outpatient,
                    col_to_analyse='AttendingPhysician',
                    title='Top 50 Outpatient Attending Physicians',
                    top_val=50,
                    color='orange')
```



In [302…

```
# Analyse Operating  Physicians in Outpatient Data

analyse_cat_columns(data_outpatient,
                    col_to_analyse='OperatingPhysician',
                    title='Top 50 Outpatient Operating Physicians',
                    top_val=50,
                    color='orange')
```



Conclusion

Most patients are operated by physician PHY330576 and attending provider is also same which a legit scenario is Outpatient claims

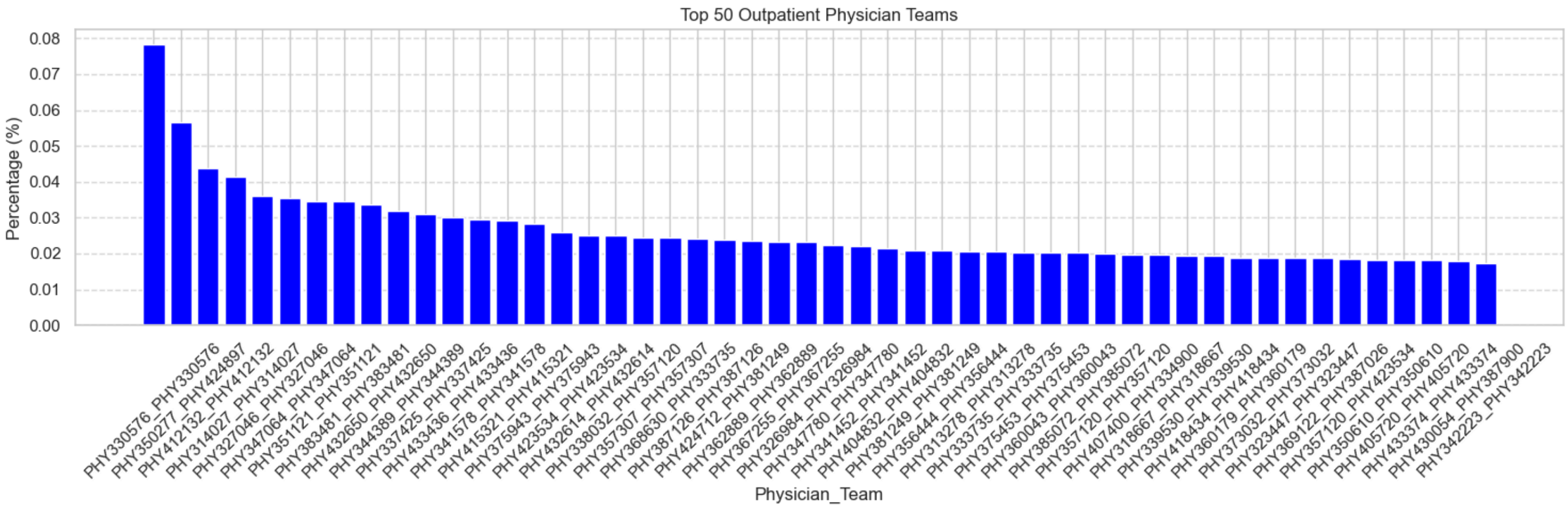
In [303…

```
# Analyzing the relationship between these two roles allows us to detect sophisticated fraud schemes:
# An Attending Physician consistently refers patients to the same Operating Physician for unnecessary surgeries.

# To analyze these together, we can create a "Pairing" feature in Python. This helps find "Closed Loops"
#  where two doctors are working together suspiciously often.

# Also for Outpatient claims mostly Attanding and Operating physicians are same.

# Create a unique 'Team' ID for every claim
data_outpatient['Physician_Team'] = (
    data_outpatient['AttendingPhysician'] + "_" + data_outpatient['OperatingPhysician']
)

# Find the most common teams
top_teams = data_outpatient['Physician_Team'].value_counts().head(10)
print(top_teams)
```

```
Physician_Team
PHY330576_PHY330576    406
PHY350277_PHY424897    293
PHY412132_PHY412132    227
PHY314027_PHY314027    214
PHY327046_PHY327046    186
PHY347064_PHY347064    183
PHY351121_PHY351121    179
PHY383481_PHY383481    179
PHY432650_PHY432650    174
PHY344389_PHY344389    165
Name: count, dtype: int64
```

In [304…

```
analyse_cat_columns(data_outpatient,
                    col_to_analyse='Physician_Team',
                    title='Top 50 Outpatient Physician Teams',
                    top_val=50,
                    color='blue')
```



Conclusion

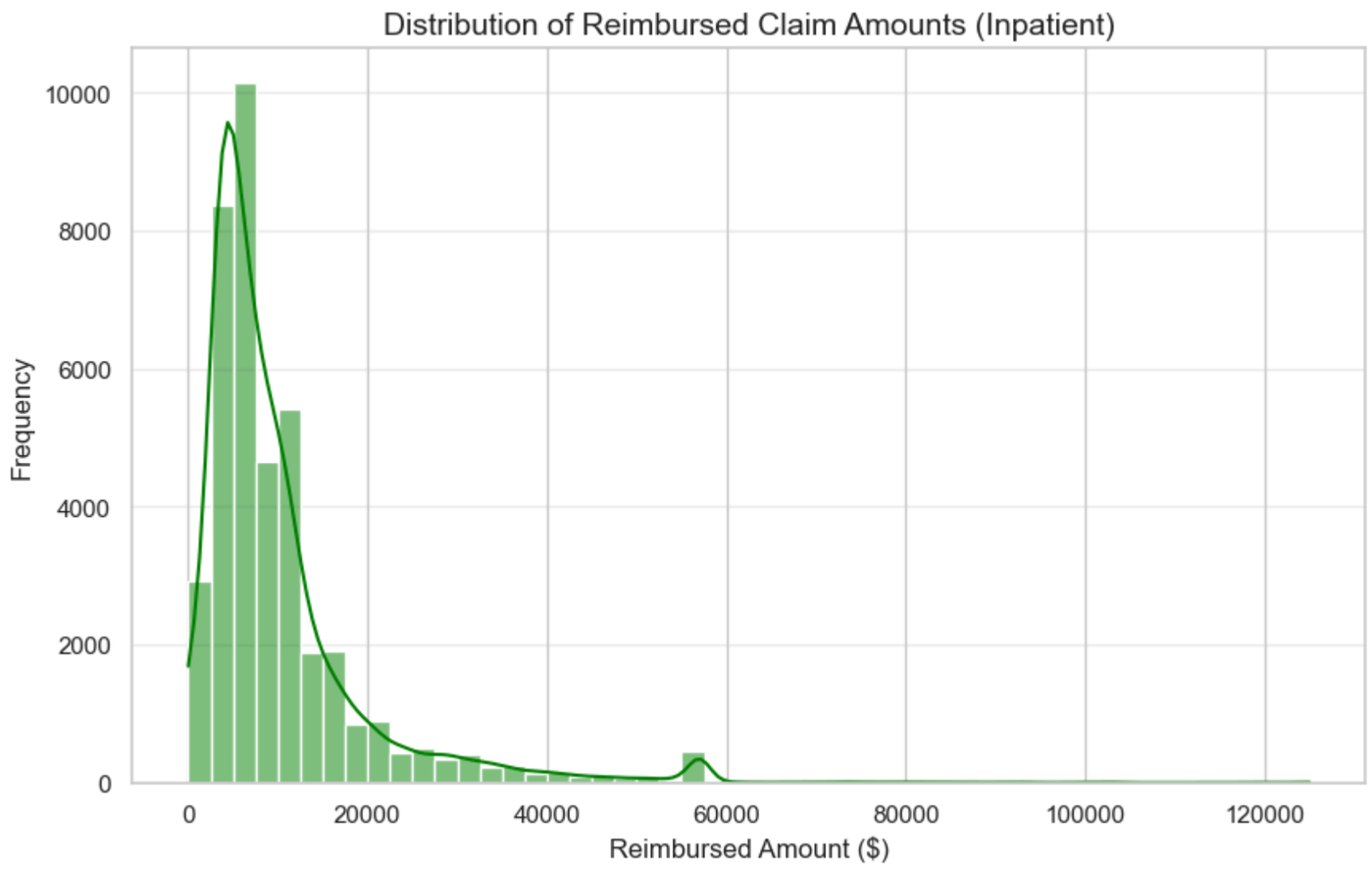
The top combination is legit one where attending and operating physician providers are same. For others where they are different data needs to be analyzed.

3 . Analysis of Claim Amount Reimbursed

In [305…

```
# Checking distribution of Claim amount for Inpatient Claims

plt.figure(figsize=(10, 6))
# Using histplot for a more modern, detailed view
```

```
sns.histplot(data_inpatient['InscClaimAmtReimbursed'], kde=True, color='green', bins=50)

plt.title('Distribution of Reimbursed Claim Amounts (Inpatient)', fontsize=14)
plt.xlabel('Reimbursed Amount ($)', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.grid(axis='y', alpha=0.3)
plt.show()
```



Conclusion

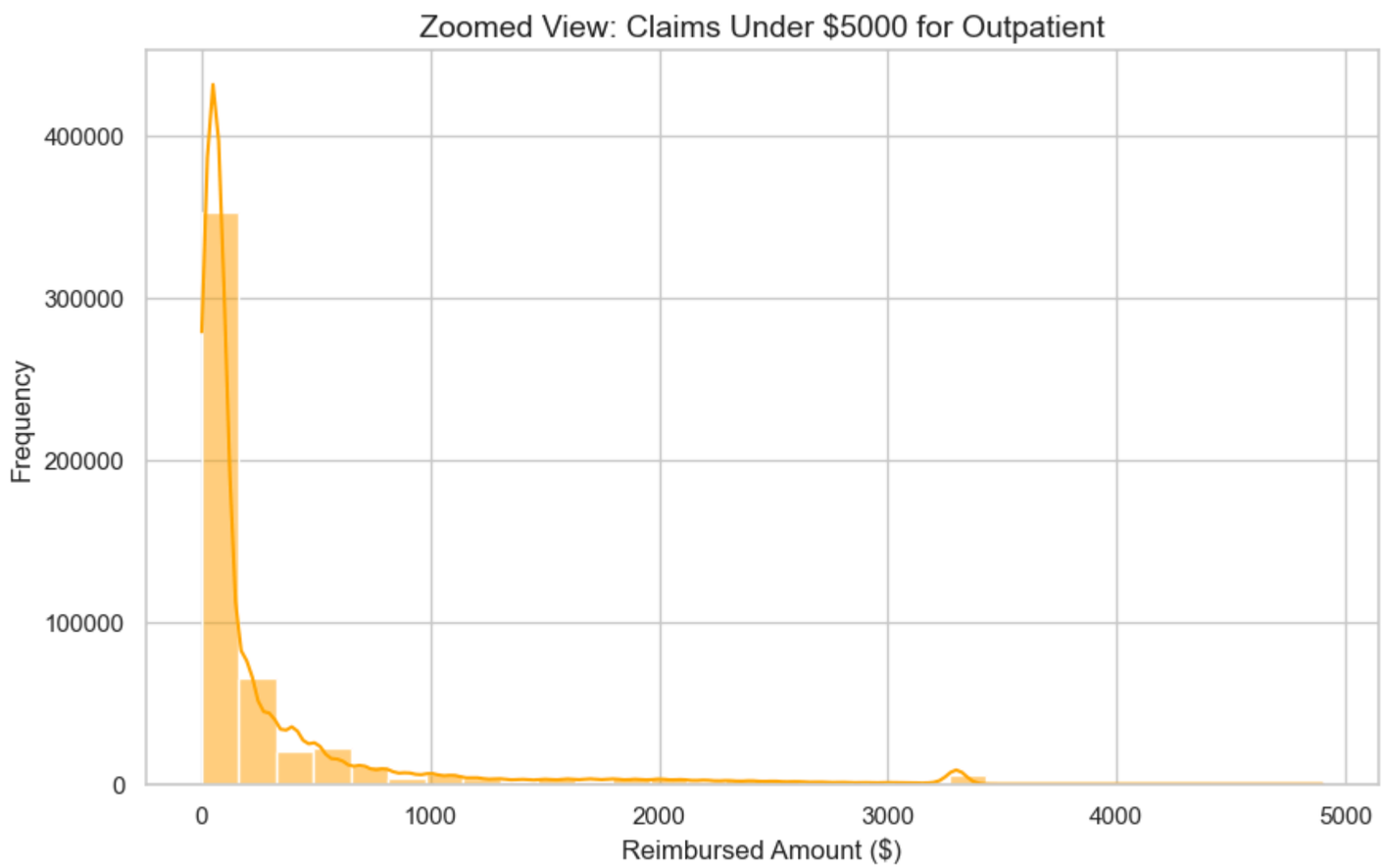
Right Skewness: Amount distribution is heavily "right-skewed," meaning the majority of claims are for small amounts, with a few very expensive outliers (like major surgeries).

Outliers: Claims that fall far to the right of the tail are the "High-Dollar" risks. These require the most scrutiny because a single fraudulent high-dollar claim can cause more damage than hundreds of small ones.

In [306…

```
# Checking distribution of Claim amount for Outpatient Claims

# Create a temporary view of only small claims
small_claims = data_outpatient[data_outpatient['InscClaimAmtReimbursed'] < 5000]

plt.figure(figsize=(10, 6))
sns.histplot(small_claims['InscClaimAmtReimbursed'], kde=True, color='orange', bins=30)

plt.title('Zoomed View: Claims Under $5000 for Outpatient', fontsize=14)
```

```
plt.xlabel('Reimbursed Amount ($)')
plt.ylabel('Frequency')
plt.show()
```



Conclusion

The distribution of Outpatient Reimbursed Amounts exhibits extreme right-skewness, with the vast majority of claims falling below $1000. The presence of claims reaching up to 3000$ indicates a small subset of high-cost outpatient procedures. For FWA detection, these high-dollar outliers should be cross-referenced with Provider IDs to determine if a single provider is disproportionately responsible for these expensive claims

Here are the high-level conclusions we can derive by connecting three data points (codes, amount and Provider): (which can change in future based on further analysis)

Identifying "Upcoding" Patterns (Codes + Amounts) - By correlating Procedure Codes with the Reimbursed Amount, we can determine if a provider is billing for high-intensity services (like complex surgeries) when the diagnosis only supports a minor intervention.

Conclusion: If the average reimbursement for a specific provider is significantly higher than the peer mean for the same procedure code, it indicates potential Upcoding or the unbundling of services to inflate the claim.

Detection of "Impossible Volume" (Provider + Claims) - Analysis of the top-billing providers (like "Super-Biller" PHY422134) allows us to perform a capacity check.

Conclusion: If a provider's claim count exceeds the 99th percentile of their peer group, they are a high-risk outlier. If the number of procedures billed in a 24-hour window exceeds human capacity, we have a strong indicator of Phantom Billing (billing for services never rendered).

Specialization Anomaly (Codes + Provider) - By looking at which codes a specific provider uses most, we can spot "out-of-specialty" billing.

Conclusion: If a General Practitioner (Attending Physician) is consistently appearing as the Operating Physician for highly specialized codes (like 3722 - Heart Surgery), it flags a major Credentialing Fraud or Scope of Practice violation.

We will not try to go in details of service codes, procedure code and specialty related to Provider. Our focus will be based on the dataset we have. But this also can be check to reduce FWA in Healthcare industry.

Term Project Milestone 2: Data Preparation

Here is a list of steps to consider performing in Milestone 2:

- Drop any features that are not useful for your model building and explain why they are not useful.
- Perform any data extraction/selection steps.
- Transform features if necessary.
- Engineer new useful features.
- Deal with missing data (do not just drop rows or columns without justifying this).
- Create dummy variables if necessary.

We need to complete some more EDA for the project

4 . Analysis of Labeled data

In [307…

```
#check all the columns of the labeled  dataset and top 5 data points
print('Columns in this dataset are : ')
print(data.columns)

data.head(5)
```

```
Columns in this dataset are :
Index(['Provider', 'PotentialFraud'], dtype='object')
```

Out[307…

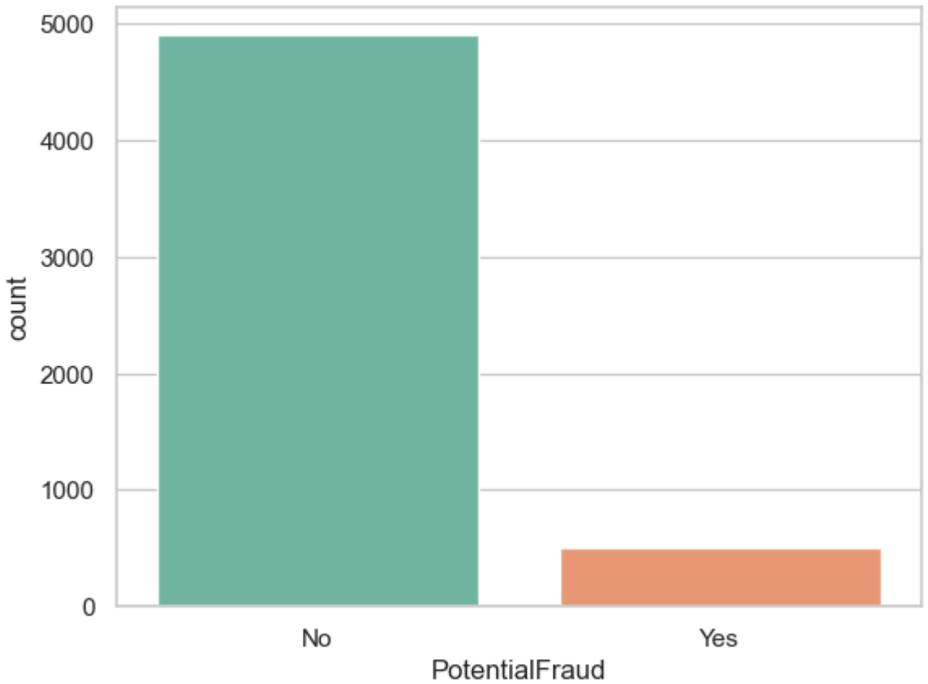
	Provider	PotentialFraud
0	PRV51001	No
1	PRV51003	Yes
2	PRV51004	No
3	PRV51005	Yes
4	PRV51007	No

In [308…

```
# Check the distribution of the target variable
sns.countplot(x='PotentialFraud',data=data, palette='Set2')
```

Out[308…

```
<Axes: xlabel='PotentialFraud', ylabel='count'>
```



Conclusion

Dataset is imbalanced

5a . Percentage of fraudulent encounters Inpatient / Outpatient

In [309…

```
# Now we will filter out only the fraudulent providers to analyze their characteristics in more detail.

only_fraud_provider = data.loc[data['PotentialFraud']=='Yes']
print("Dataset shape : ", only_fraud_provider.shape)
only_fraud_provider.head(5)
```

```
Dataset shape :  (506, 2)
```

Out[309…

	Provider	PotentialFraud
1	PRV51003	Yes
3	PRV51005	Yes
13	PRV51021	Yes
25	PRV51037	Yes
37	PRV51052	Yes

In [310…

```
# Now we will merge this dataset with the Inpatient and Outpatient datasets to analyze the characteristics of fraudulent providers in more detail.

fraud_provider_inpatient_df = pd.merge(data_inpatient, only_fraud_provider, how='inner', on='Provider')
```

```
print("Dataset shape : ", fraud_provider_inpatient_df.shape)
fraud_provider_inpatient_df.head(5)
```

```
Dataset shape :  (23402, 31)
```

Out[310…

	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	Provider	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician	OtherPhysician	AdmissionDt	...	ClmDiagnosisCode_8	ClmDiagno
0	BENE11001	CLM46614	2009-04-12	2009-04-18	PRV55912	26000	PHY390922	NaN	NaN	2009-04-12	...	19889	
1	BENE11017	CLM70950	2009-10-06	2009-10-12	PRV54986	8000	PHY402711	PHY402711	PHY402711	2009-10-06	...	25000	
2	BENE11028	CLM62376	2009-08-03	2009-08-07	PRV51148	6000	PHY346286	PHY405514	NaN	2009-08-03	...	4439	
3	BENE11034	CLM31519	2008-12-29	2009-01-05	PRV55215	29000	PHY355604	PHY415867	NaN	2008-12-29	...	4111	
4	BENE11034	CLM57949	2009-07-01	2009-07-09	PRV55193	102000	PHY397979	PHY418257	NaN	2009-07-01	...	591	

5 rows × 31 columns

In [311…

```
# Now we will check what percentage of the inpatient data is related to fraudulent providers.

print("Dataset shape : ", fraud_provider_inpatient_df.shape)
print("Percentage of fraud cases related to inpatient data : ",(fraud_provider_inpatient_df.shape[0]/data_inpatient.shape[0])*100)
```

```
Dataset shape :  (23402, 31)
Percentage of fraud cases related to inpatient data :  57.81983495577408
```

Conclusion

Around 58 % of the inpatient data for training has fraudulent encounter

In [312…

```
# Now we will check what percentage of the outpatient data is related to fraudulent providers.

fraud_provider_outpatient_df = pd.merge(data_outpatient, only_fraud_provider, how='inner', on='Provider')
print("Dataset shape : ", fraud_provider_outpatient_df.shape)
fraud_provider_outpatient_df.head(5)
```

```
Dataset shape :  (189394, 29)
```

Out[312…

	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	Provider	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician	OtherPhysician	ClmDiagnosisCode_1	...	ClmProcedureCode_1	C
0	BENE11002	CLM624349	2009-10-11	2009-10-11	PRV56011	30	PHY326117	NaN	NaN	78943	...	NaN	
1	BENE11004	CLM121801	2009-01-06	2009-01-06	PRV56011	40	PHY334319	NaN	NaN	71988	...	NaN	
2	BENE11004	CLM150998	2009-01-22	2009-01-22	PRV56011	200	PHY403831	NaN	NaN	82382	...	NaN	
3	BENE11004	CLM173224	2009-02-03	2009-02-03	PRV56011	20	PHY339887	NaN	NaN	20381	...	NaN	
4	BENE11004	CLM224741	2009-03-03	2009-03-03	PRV56011	40	PHY345721	NaN	NaN	V6546	...	NaN	

5 rows × 29 columns

In [313…

```
# Now we will check what percentage of the outpatient data is related to fraudulent providers.

print("Dataset shape : ", fraud_provider_outpatient_df.shape)
print("Percentage of fraud cases related to outpatient data : ",(fraud_provider_outpatient_df.shape[0]/data_outpatient.shape[0])*100)
```

```
Dataset shape :  (189394, 29)
Percentage of fraud cases related to outpatient data :  36.58112130290089
```

Conclusion

Around 37 % of the outpatient data for training has fraudulent encounter

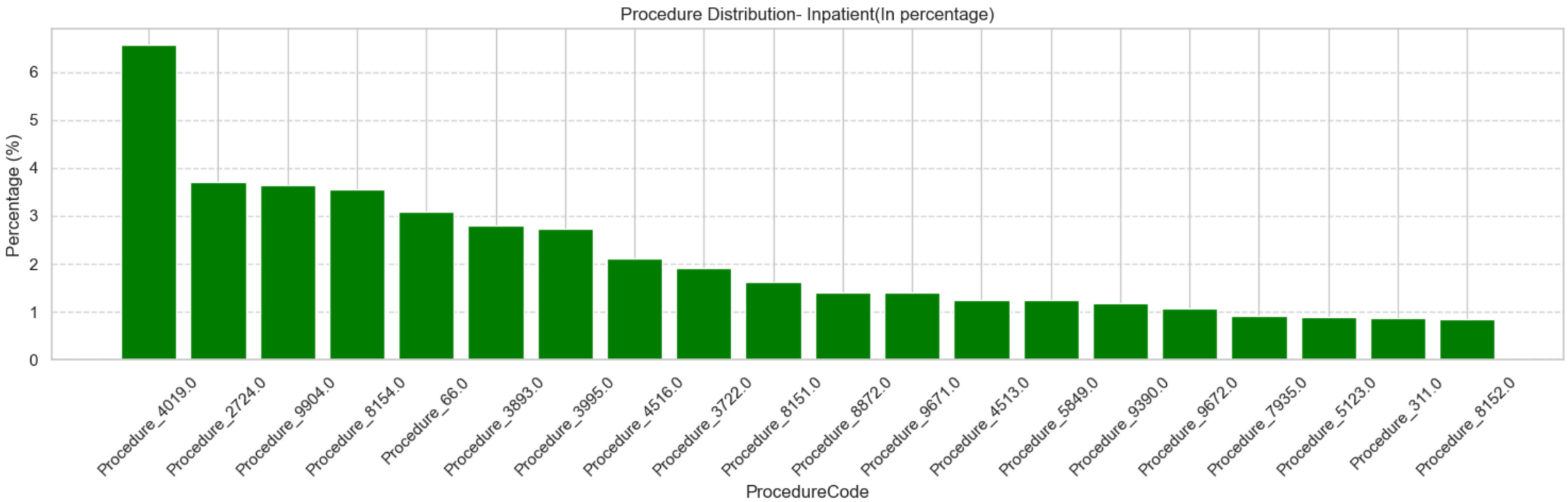
5b . 20 Most used procedure codes and diagnosis codes used by the potential fraudulent providers

Inpatient - Procedure

In [314…

```
#create a dataframe for concatenating all Procedure Code columns

inpatient_df = pd.DataFrame(columns = ['ProcedureCode'])
inpatient_df['ProcedureCode'] = pd.concat([fraud_provider_inpatient_df["ClmProcedureCode_1"],
                                           fraud_provider_inpatient_df["ClmProcedureCode_2"],
                                           fraud_provider_inpatient_df["ClmProcedureCode_3"],
                                           fraud_provider_inpatient_df["ClmProcedureCode_4"],
                                           fraud_provider_inpatient_df["ClmProcedureCode_5"],
                                           fraud_provider_inpatient_df["ClmProcedureCode_6"]], axis=0, sort=True).dropna()

print(inpatient_df.shape)

analyse_cat_columns(inpatient_df,col_to_analyse='ProcedureCode' , prefix = 'Procedure_',title = 'Procedure Distribution- Inpatient(In percentage)',top_val = 20)
```

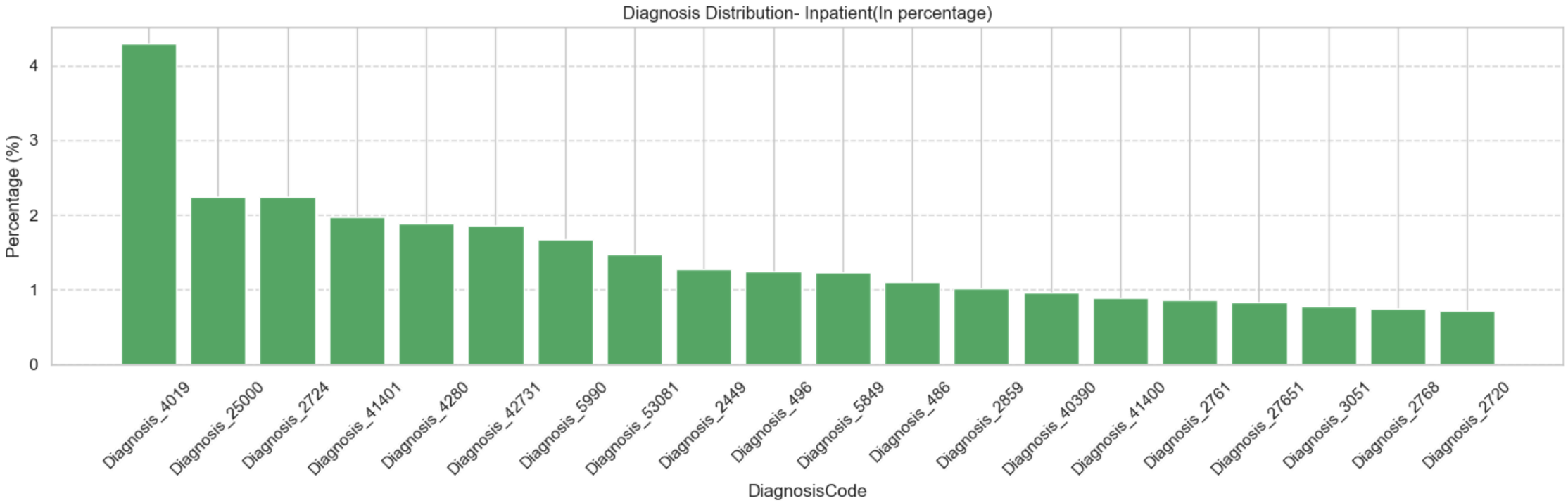
```
(17325, 1)
```



In [315…

```
#create a dataframe for concatenating all DiagnosisCode Code columns

inpatient_df = pd.DataFrame(columns = ['DiagnosisCode'])
inpatient_df['DiagnosisCode'] = pd.concat([fraud_provider_inpatient_df["ClmDiagnosisCode_1"],
                                           fraud_provider_inpatient_df["ClmDiagnosisCode_2"],
                                           fraud_provider_inpatient_df["ClmDiagnosisCode_3"],
                                           fraud_provider_inpatient_df["ClmDiagnosisCode_4"],
                                           fraud_provider_inpatient_df["ClmDiagnosisCode_5"],
                                           fraud_provider_inpatient_df["ClmDiagnosisCode_6"],
                                           fraud_provider_inpatient_df["ClmDiagnosisCode_7"],
                                           fraud_provider_inpatient_df["ClmDiagnosisCode_8"],
                                           fraud_provider_inpatient_df["ClmDiagnosisCode_9"],
                                           fraud_provider_inpatient_df["ClmDiagnosisCode_10"]], axis=0, sort=True).dropna()

print(inpatient_df.shape)

analyse_cat_columns(inpatient_df,col_to_analyse='DiagnosisCode' , prefix = 'Diagnosis_',title = 'Diagnosis Distribution- Inpatient(In percentage)',top_val = 20 ,color = 'g')
```

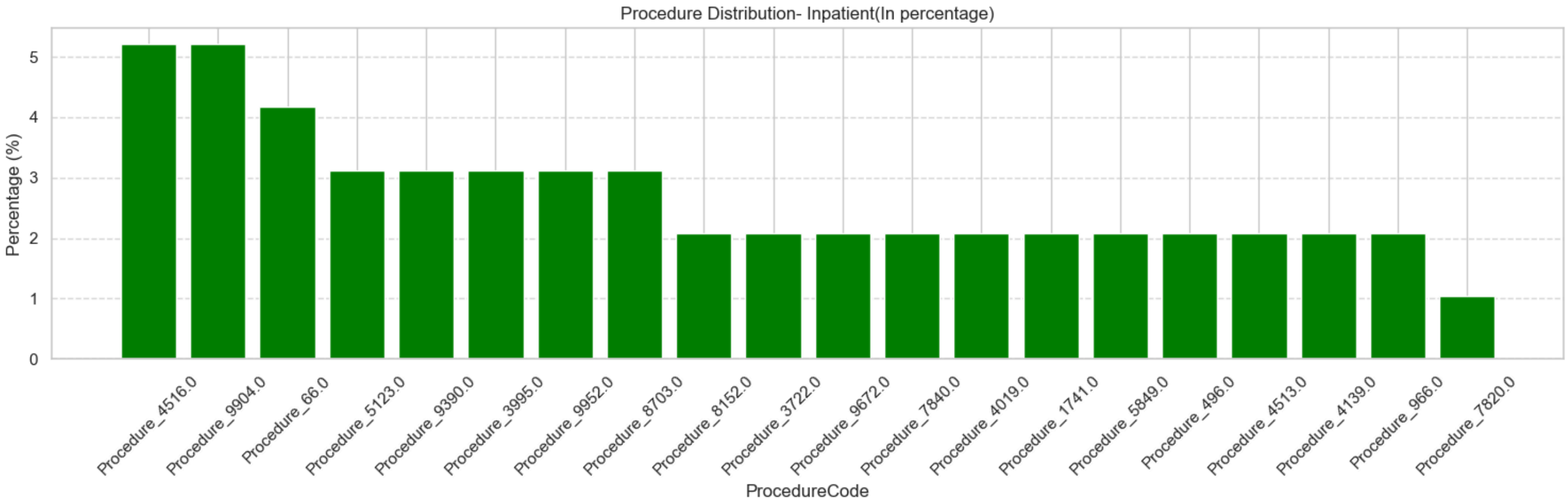
```
(189329, 1)
```



In [316…

```
#create a dataframe for concatenating all Procedure Code columns

inpatient_df = pd.DataFrame(columns = ['ProcedureCode'])
inpatient_df['ProcedureCode'] = pd.concat([fraud_provider_outpatient_df["ClmProcedureCode_1"],
                                           fraud_provider_outpatient_df["ClmProcedureCode_2"],
                                           fraud_provider_outpatient_df["ClmProcedureCode_3"],
                                           fraud_provider_outpatient_df["ClmProcedureCode_4"],
                                           fraud_provider_inpatient_df["ClmProcedureCode_5"],
                                           fraud_provider_outpatient_df["ClmProcedureCode_6"]], axis=0, sort=True).dropna()

print(inpatient_df.shape)

analyse_cat_columns(inpatient_df,col_to_analyse='ProcedureCode' , prefix = 'Procedure_',title = 'Procedure Distribution- Inpatient(In percentage)',top_val = 20)
```

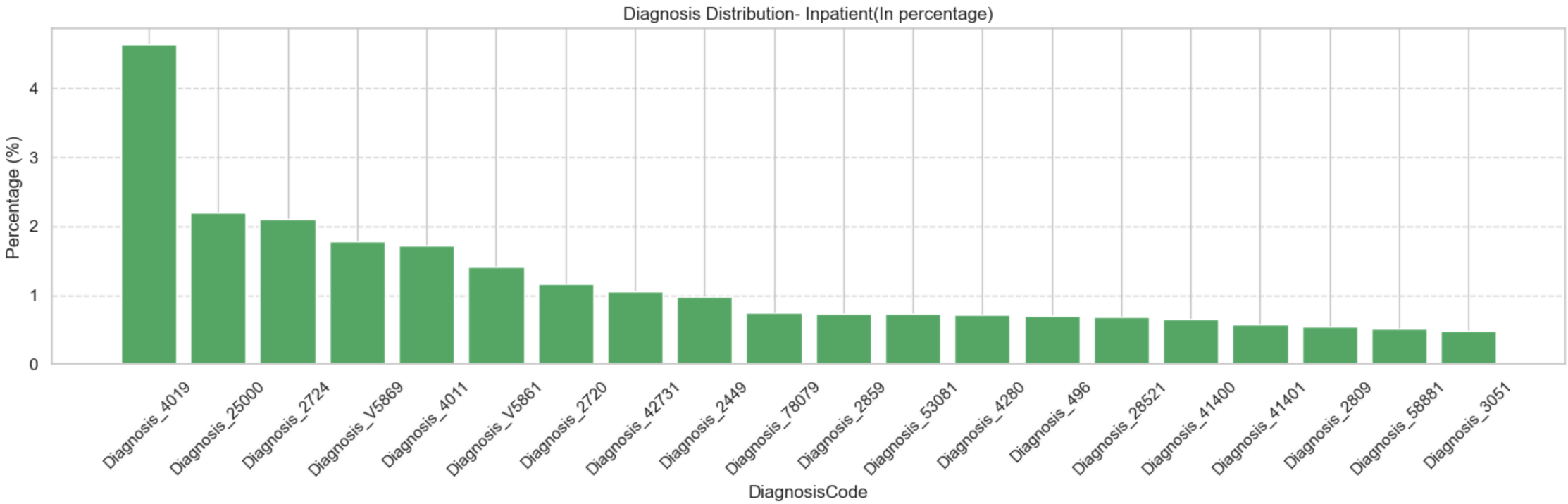
```
(96, 1)
```



In [317…

```
#create a dataframe for concatenating all DiagnosisCode Code columns

inpatient_df = pd.DataFrame(columns = ['DiagnosisCode'])
inpatient_df['DiagnosisCode'] = pd.concat([fraud_provider_outpatient_df["ClmDiagnosisCode_1"],
                                           fraud_provider_outpatient_df["ClmDiagnosisCode_2"],
                                           fraud_provider_outpatient_df["ClmDiagnosisCode_3"],
                                           fraud_provider_outpatient_df["ClmDiagnosisCode_4"],
                                           fraud_provider_outpatient_df["ClmDiagnosisCode_5"],
                                           fraud_provider_outpatient_df["ClmDiagnosisCode_6"],
                                           fraud_provider_outpatient_df["ClmDiagnosisCode_7"],
                                           fraud_provider_outpatient_df["ClmDiagnosisCode_8"],
                                           fraud_provider_outpatient_df["ClmDiagnosisCode_9"],
                                           fraud_provider_outpatient_df["ClmDiagnosisCode_10"]], axis=0, sort=True).dropna()

print(inpatient_df.shape)

analyse_cat_columns(inpatient_df,col_to_analyse='DiagnosisCode' , prefix = 'Diagnosis_',title = 'Diagnosis Distribution- Inpatient(In percentage)',top_val = 20 ,color = 'g')
```

```
(494869, 1)
```



6. Gaining Insights from beneficiary Data

In [318…

```
#check all the columns of the beneficiary dataset and top 5 data points

print('Columns in this dataset are : ')
print(data_beneficiary.columns)

data_beneficiary.head(2)
```

```
Columns in this dataset are :
Index(['BeneID', 'DOB', 'DOD', 'Gender', 'Race', 'RenalDiseaseIndicator',
       'State', 'County', 'NoOfMonths_PartACov', 'NoOfMonths_PartBCov',
       'ChronicCond_Alzheimer', 'ChronicCond_Heartfailure',
       'ChronicCond_KidneyDisease', 'ChronicCond_Cancer',
       'ChronicCond_ObstrPulmonary', 'ChronicCond_Depression',
       'ChronicCond_Diabetes', 'ChronicCond_IschemicHeart',
       'ChronicCond_Osteoporasis', 'ChronicCond_rheumatoidarthritis',
       'ChronicCond_stroke', 'IPAnnualReimbursementAmt',
       'IPAnnualDeductibleAmt', 'OPAnnualReimbursementAmt',
       'OPAnnualDeductibleAmt'],
      dtype='object')
```

Out[318…

	BeneID	DOB	DOD	Gender	Race	RenalDiseaseIndicator	State	County	NoOfMonths_PartACov	NoOfMonths_PartBCov	…	ChronicCond_Depression	ChronicCond_Diabetes	ChronicCond_Is
0	BENE11001	1943-01-01	NaN	1	1	0	39	230	12	12	…	1	1	
1	BENE11002	1936-09-01	NaN	2	1	0	39	280	12	12	…	2	2	

2 rows × 25 columns

6a . State and County

In [319…

```
# Now we will merge the beneficiary dataset with the inpatient dataset to analyze the characteristics of beneficiaries related to
# fraudulent providers in more detail.

bene_inpat_df = pd.merge(data_beneficiary, fraud_provider_inpatient_df, how='inner', on='BeneID')
print(bene_inpat_df.shape)
bene_inpat_df.head(5)
```

```
(23402, 55)
```

Out[319…

	BeneID	DOB	DOD	Gender	Race	RenalDiseaseIndicator	State	County	NoOfMonths_PartACov	NoOfMonths_PartBCov	…	ClmDiagnosisCode_8	ClmDiagnosisCode_9	ClmDiagnosisCode_1(
0	BENE11001	1943-01-01	NaN	1	1	0	39	230	12	12	…	19889	5849	NaN
1	BENE11017	1940-06-01	NaN	2	1	0	31	270	12	12	…	25000	25002	NaN
2	BENE11028	1941-12-01	NaN	1	1	0	38	230	12	12	…	4439	41401	NaN
3	BENE11034	1946-03-01	NaN	2	1	0	34	760	12	12	…	4111	4589	NaN
4	BENE11034	1946-03-01	NaN	2	1	0	34	760	12	12	…	591	51881	NaN

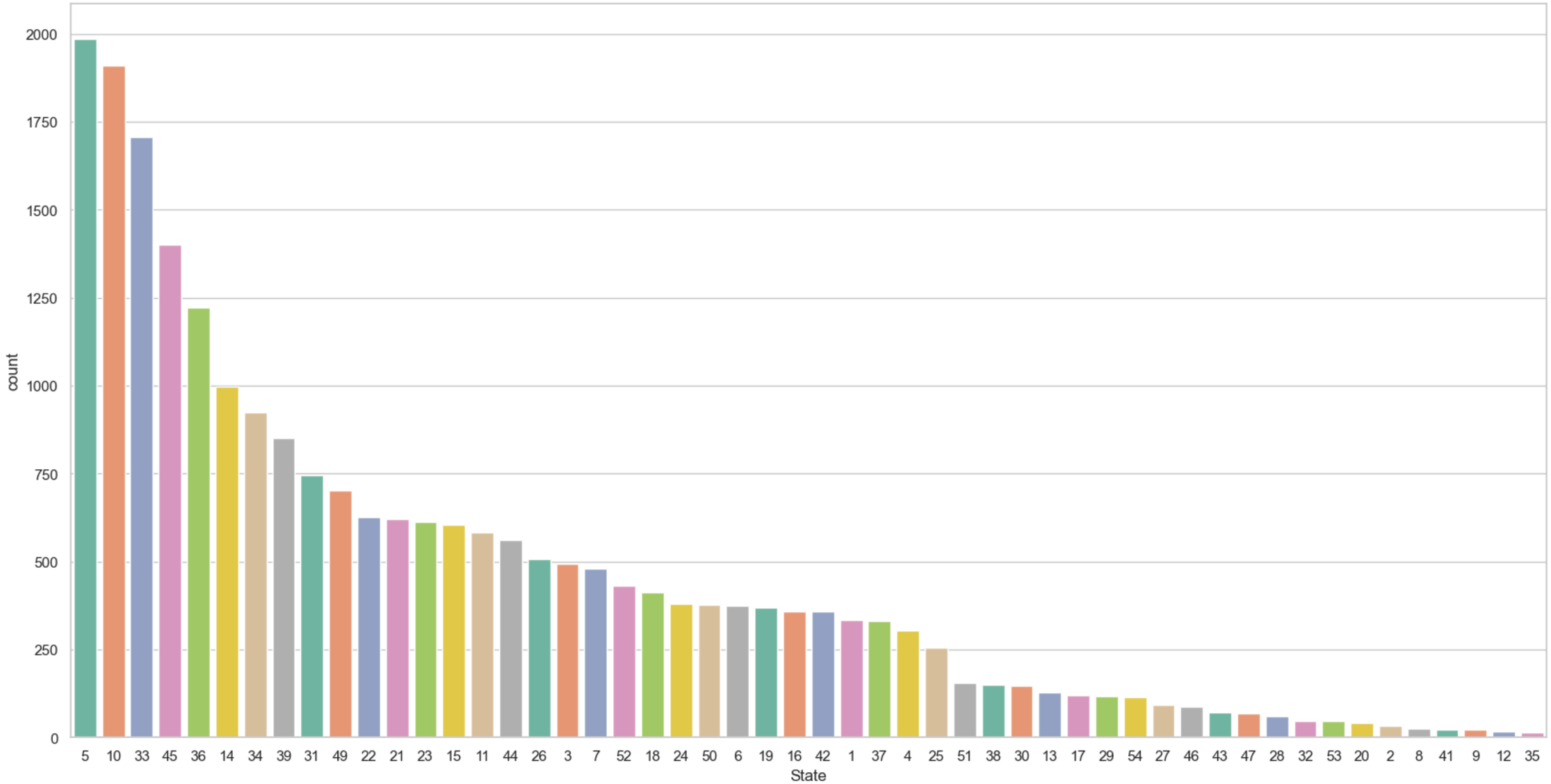
5 rows × 55 columns

In [320…

```
# Now we will merge the beneficiary dataset with the inpatient dataset to analyze the characteristics of beneficiaries related to
# fraudulent providers in more detail.

plt.figure(figsize=(20, 10))
sns.countplot(x='State',y=None, data=bene_inpat_df, orient ="v", order = bene_inpat_df['State'].value_counts().index, palette='Set2')
```

Out[320…

```
<Axes: xlabel='State', ylabel='count'>
```

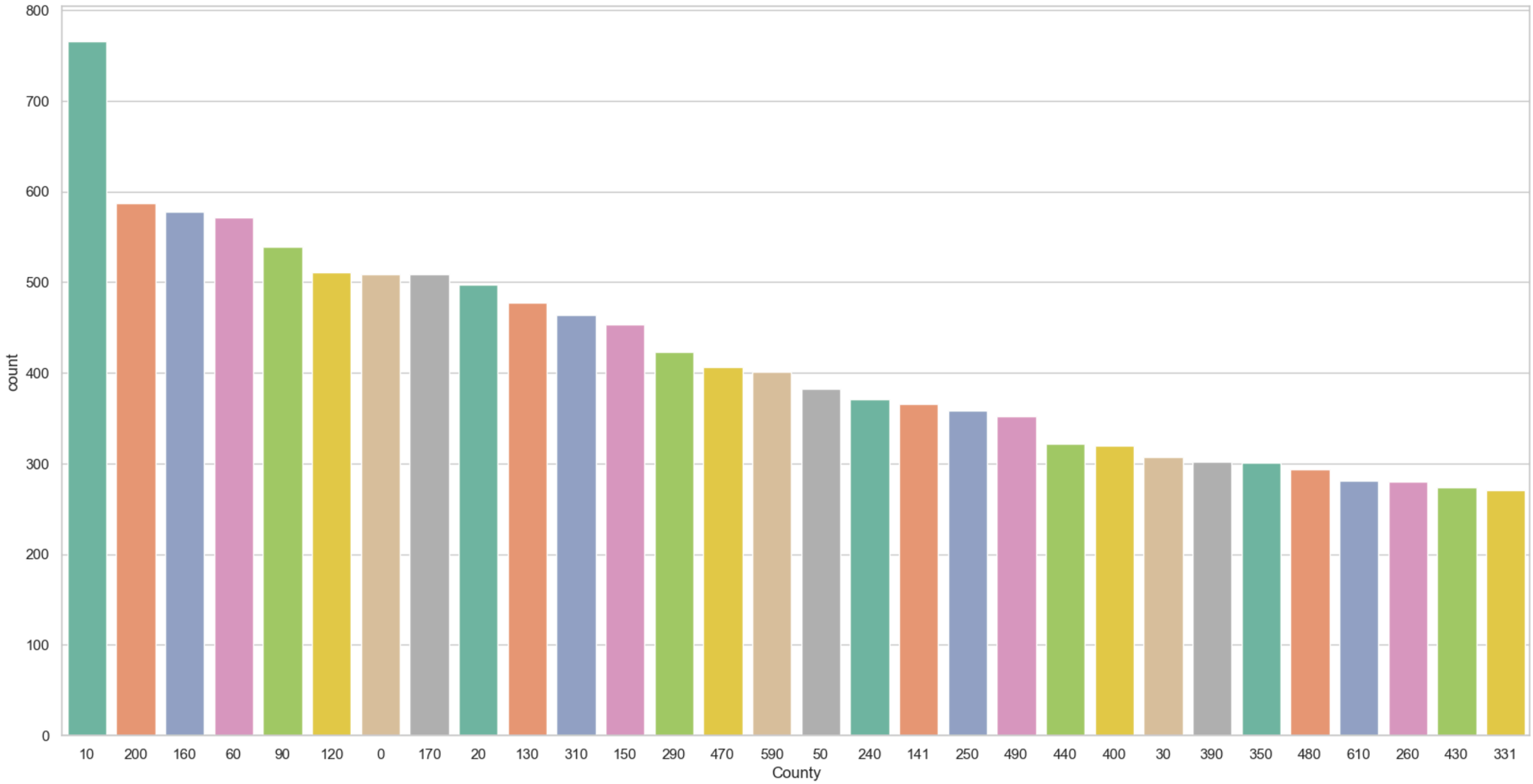


In [321…

```
# Now we will merge the beneficiary dataset with the inpatient dataset to analyze the characteristics of beneficiaries related to
# fraudulent providers in more detail.
plt.figure(figsize=(20, 10))
sns.countplot(x='County',y=None, data=bene_inpat_df, orient ="v", order = bene_inpat_df['County'].value_counts().head(30).index, palette='Set2')
```

Out[321…

```
<Axes: xlabel='County', ylabel='count'>
```



Conclusion

Inpatient + fraud -- State

State coded as 5,10,33 and 45 have most fraudulent encounters for Inpatient data . County coded as 10,200,160 and 60 have most fraudulent encounters for Inpatient data .

Outpatient + Fraud -- State

In [322…

```
# Now we will merge the beneficiary dataset with the outpatient dataset to analyze the characteristics of beneficiaries related to
# fraudulent providers in more detail.

bene_outpat_df = pd.merge(data_beneficiary, fraud_provider_outpatient_df, how='inner', on='BeneID')
```

```
print(bene_outpat_df.shape)
bene_outpat_df.head(5)
```

```
(189394, 53)
```

Out[322…

	BeneID	DOB	DOD	Gender	Race	RenalDiseaseIndicator	State	County	NoOfMonths_PartACov	NoOfMonths_PartBCov	...	ClmProcedureCode_1	ClmProcedureCode_2	ClmProcedureCode_3
0	BENE11002	1936-09-01	NaN	2	1	0	39	280	12	12	...	NaN	NaN	NaN
1	BENE11004	1922-07-01	NaN	1	1	0	39	270	12	12	...	NaN	NaN	NaN
2	BENE11004	1922-07-01	NaN	1	1	0	39	270	12	12	...	NaN	NaN	NaN
3	BENE11004	1922-07-01	NaN	1	1	0	39	270	12	12	...	NaN	NaN	NaN
4	BENE11004	1922-07-01	NaN	1	1	0	39	270	12	12	...	NaN	NaN	NaN

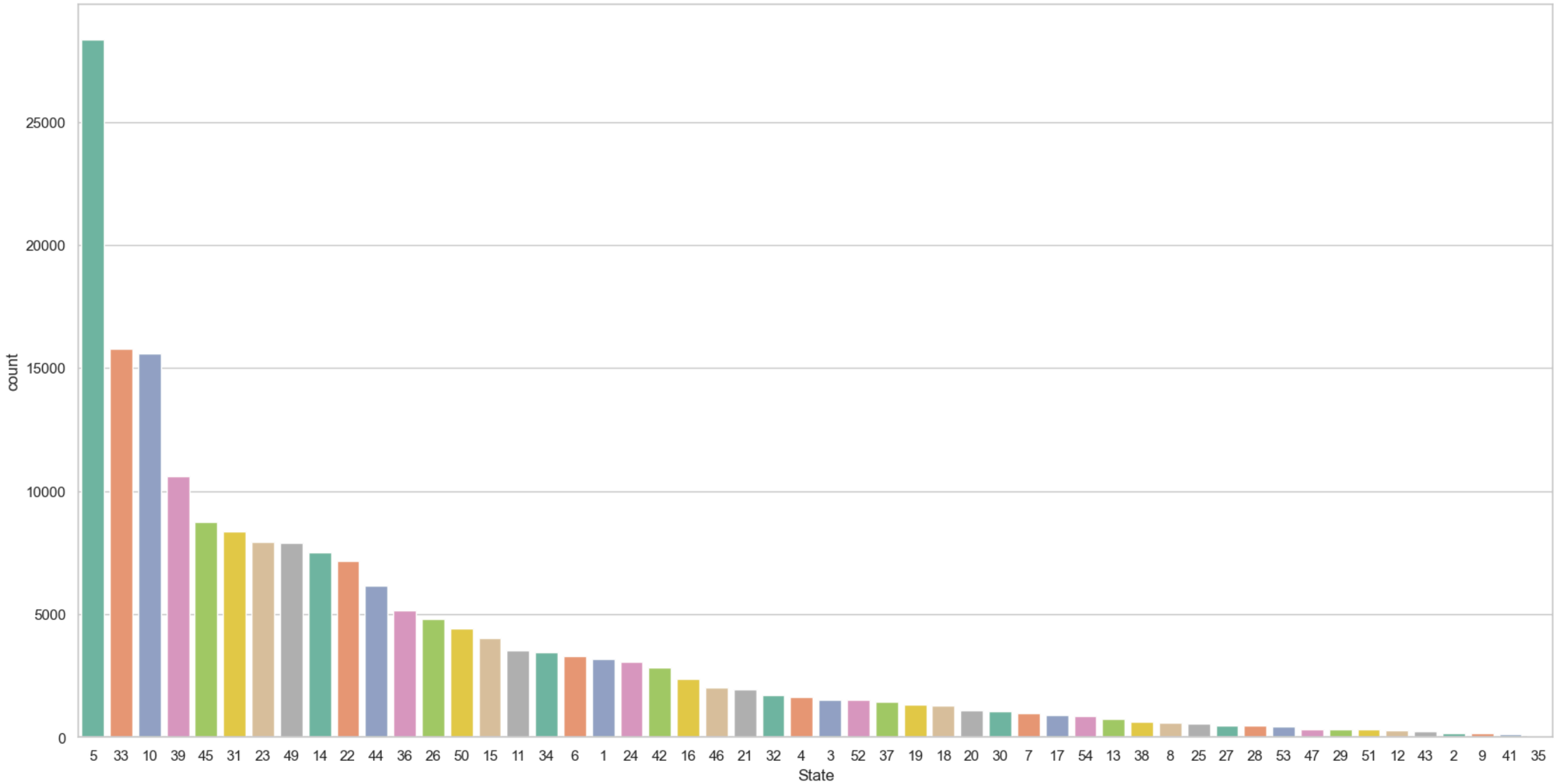
5 rows × 53 columns

In [323…

```
# Now we will merge the beneficiary dataset with the outpatient dataset to analyze the characteristics of beneficiaries related to
# fraudulent providers in more detail.

plt.figure(figsize=(20, 10))
sns.countplot(x='State',y=None, data=bene_outpat_df, orient ="v", order = bene_outpat_df['State'].value_counts().index, palette='Set2')
```

Out[323…

```
<Axes: xlabel='State', ylabel='count'>
```

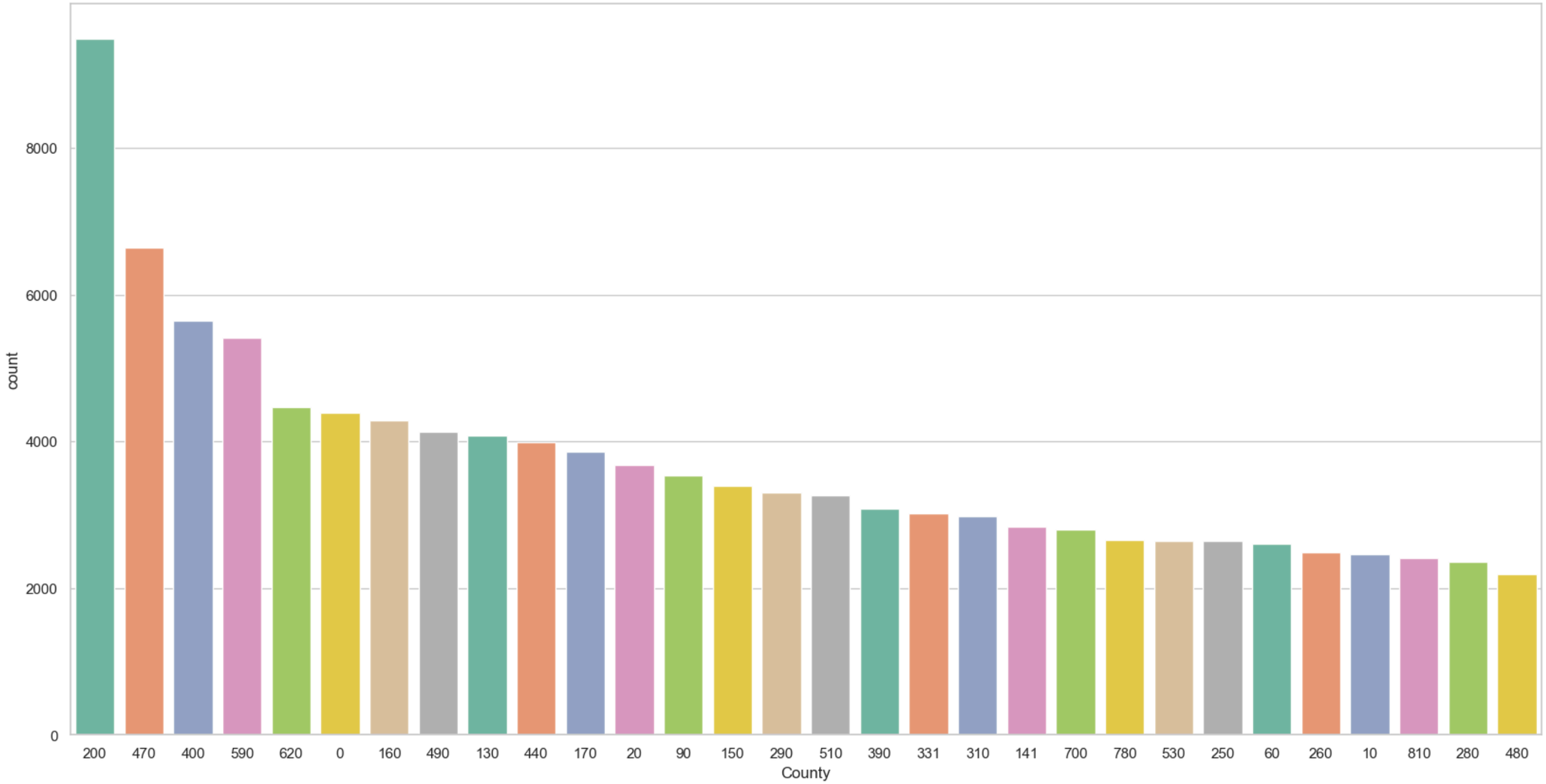


In [324…

```
# Now we will merge the beneficiary dataset with the outpatient dataset to analyze the characteristics of beneficiaries related to
# fraudulent providers in more detail.

plt.figure(figsize=(20, 10))
sns.countplot(x='County',y=None, data=bene_outpat_df, orient ="v", order = bene_outpat_df['County'].value_counts().head(30).index, palette='Set2')
```

Out[324…

```
<Axes: xlabel='County', ylabel='count'>
```



Conclusion

Outpatient + fraud -- State

State coded as 5,33,10 and 39 have most fraudulent encounters for Inpatient data. County coded as 200,470,400 and 590 have most fraudulent encounters for Outpatient data .

6b . DOB

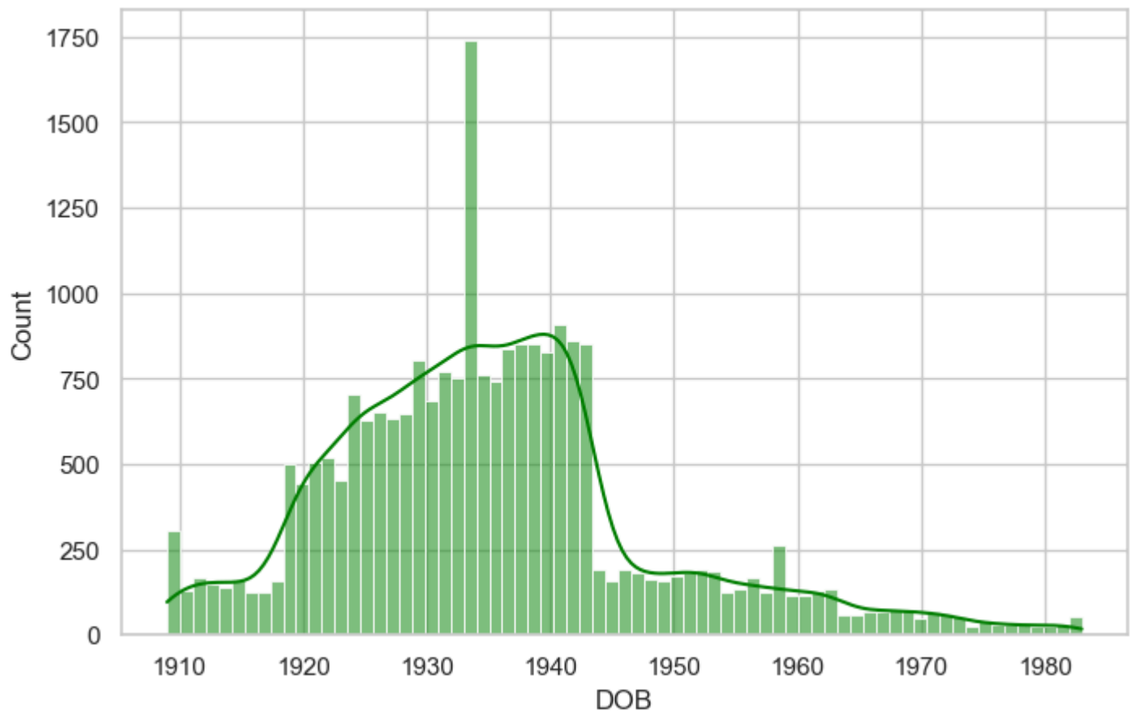
Inpatient + Fraud - DOB

In [325…

```
# Ensure DOB is a datetime type (if not already)
bene_inpat_df['DOB'] = pd.to_datetime(bene_inpat_df['DOB'])

# histogram of birth years to see the age distribution of beneficiaries associated with fraudulent providers
plt.figure(figsize=(8, 5))
sns.histplot(bene_inpat_df.DOB.apply(get_year), kde=True, color='green')
```

Out[325…

```
<Axes: xlabel='DOB', ylabel='Count'>
```



Conclusion

Inpatient + Fraud - DOB

Most Fraudulent encounters in Inpatient Data is observed for the patients born between 1920 and 1945

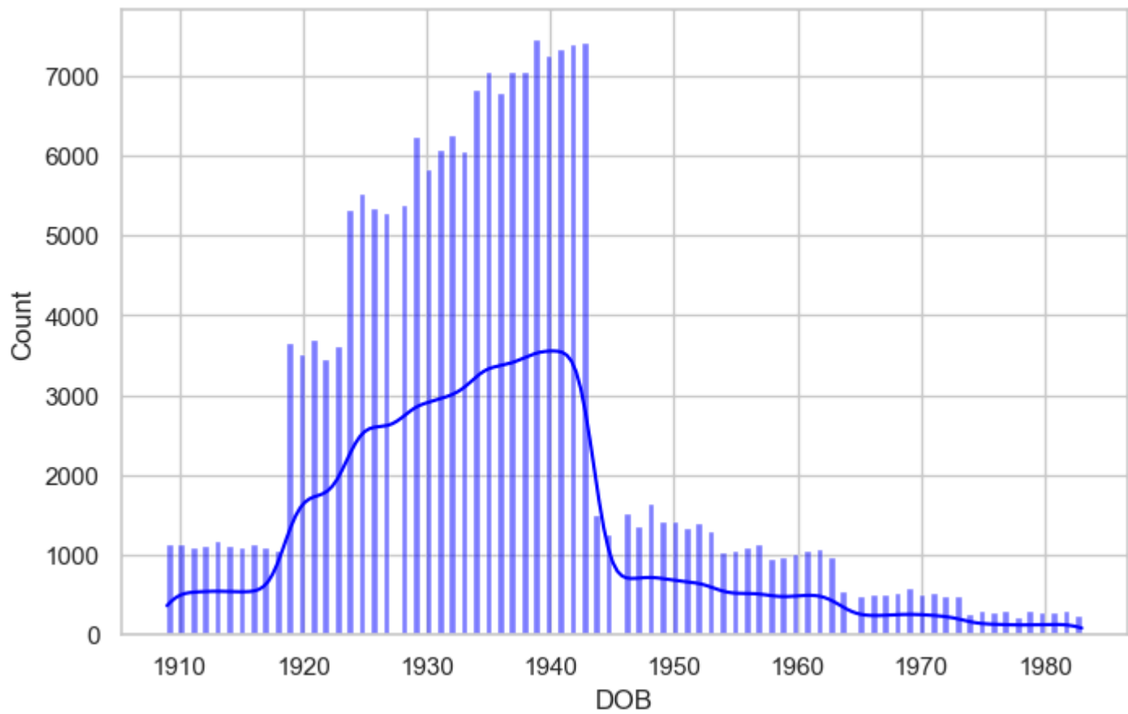
Outpatient + Fraud - DOB

In [326…

```
# Ensure DOB is a datetime type (if not already)
bene_outpat_df['DOB'] = pd.to_datetime(bene_outpat_df['DOB'])

# histogram of birth years to see the age distribution of beneficiaries associated with fraudulent providers
plt.figure(figsize=(8, 5))
sns.histplot(bene_outpat_df.DOB.apply(get_year), kde=True, color='blue')
```

Out[326…

```
<Axes: xlabel='DOB', ylabel='Count'>
```



Conclusion

Outpatient + Fraud - DOB

Most Fraudulent encounters in Inpatient Data is observed for the patients born between 1920 and 1945

7. Calculating Money lost in Fraud for both Inpatient and outpatinet data

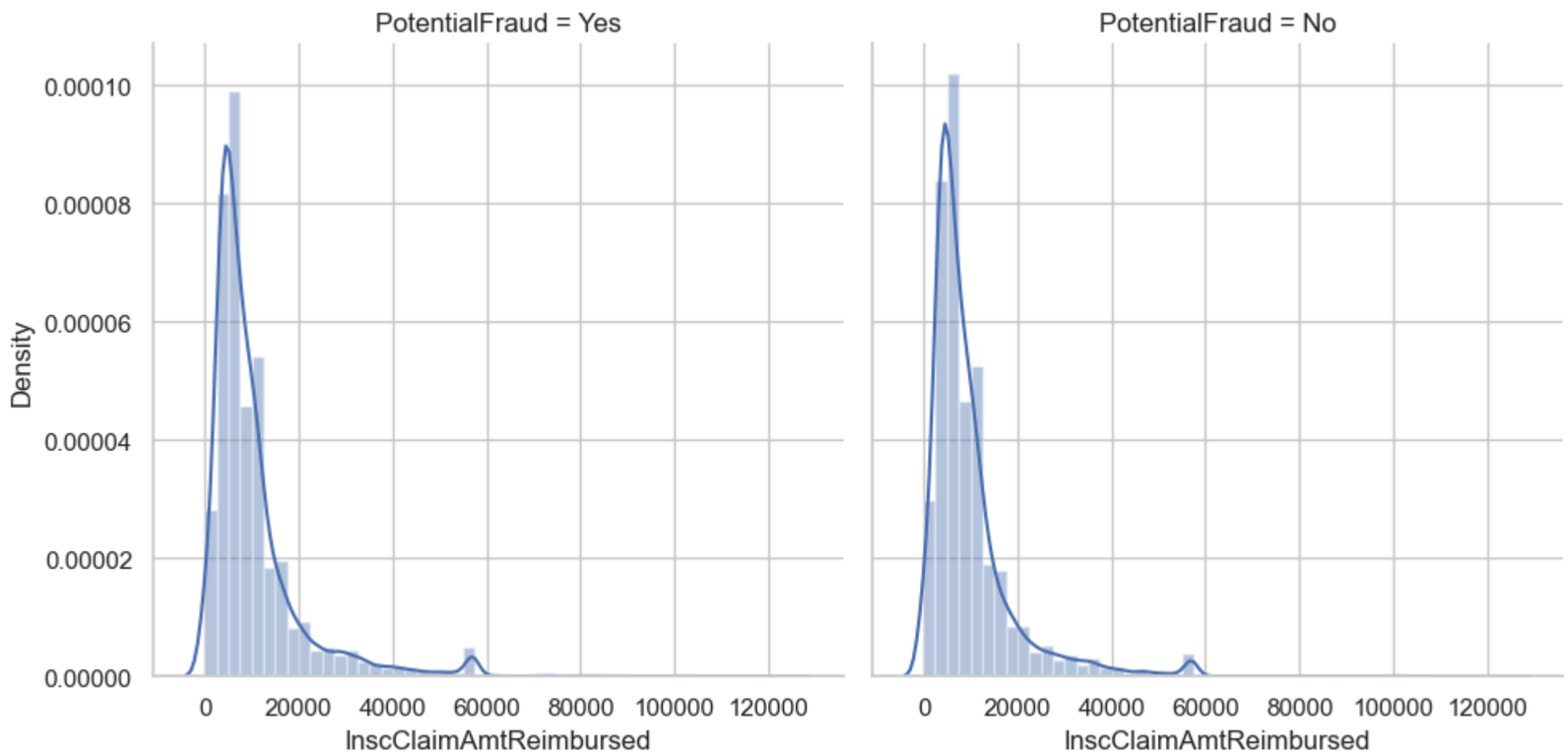
In [327…

```
# Now we will merge the beneficiary dataset with the inpatient dataset to analyze the characteristics of beneficiaries related to
# fraudulent providers in more detail.

inpat_labeled_provider_df = pd.merge(data_inpatient , data , how='inner', on='Provider')
print(inpat_labeled_provider_df.shape)
```

```
(40474, 31)
```

In [328…

```
# Now we will merge the beneficiary dataset with the inpatient dataset to analyze the characteristics of beneficiaries related to
# fraudulent providers in more detail.

sns.FacetGrid(inpat_labeled_provider_df, col='PotentialFraud',height=5).map(sns.distplot, "InscClaimAmtReimbursed",).add_legend()
plt.show()
plt.tight_layout()
```




```
<Figure size 640x480 with 0 Axes>
```

In [329…

```
# calculating total money lost inpatient fraudulent encounters

Total_money_lost = inpat_labeled_provider_df.loc[inpat_labeled_provider_df['PotentialFraud']== 'Yes'].InscClaimAmtReimbursed.sum()

print("Total money lost : ",Total_money_lost)
```

```
Total money lost :  241288510
```

In [330…

```
# calculating total money lost outpatient fraudulent encounters

outpat_labeled_provider_df = pd.merge(data_outpatient , data , how='inner', on='Provider')
print(outpat_labeled_provider_df.shape)
```

```
(517737, 29)
```

In [331…

```
# calculating total money lost in outpatient fraudulent encounters
Total_money_lost = outpat_labeled_provider_df.loc[outpat_labeled_provider_df['PotentialFraud']== 'Yes'].InscClaimAmtReimbursed.sum()

print("Total money lost : ",Total_money_lost)
```

```
Total money lost :  54392610
```

In [332…

```
print('Total money lost as per the data = ','241,288,510 + 54,392,610, That is around 295 Million' )
```

```
Total money lost as per the data =  241,288,510 + 54,392,610, That is around 295 Million
```

Till this point we have done thorough EDA on columns of each of the data files. Now we will merge them all and then perform EDA to establish a trend in data and the target.

8 . Data preprocessing and Feature Engineering

In [333…
```
# Create a binary target variable for whether the claim was admitted (inpatient) or not (outpatient)

data_inpatient['is_admitted'] = 1
data_outpatient['is_admitted'] = 0
```

In [334…
```
# Merge in_pt, out_pt and ben df into a single patient dataset
data1 = pd.merge(data_inpatient, data_outpatient,
                    left_on = [ idx for idx in data_outpatient.columns if idx in data_inpatient.columns],
                    right_on = [ idx for idx in data_outpatient.columns if idx in data_inpatient.columns],
                    how = 'outer').\
          merge(data_beneficiary,left_on='BeneID',right_on='BeneID',how='inner')
```

In [335…
```
# Replace values with a binary annotation

data1 = data1.replace({'ChronicCond_Alzheimer': 2, 'ChronicCond_Heartfailure': 2, 'ChronicCond_KidneyDisease': 2,
                   'ChronicCond_Cancer': 2, 'ChronicCond_ObstrPulmonary': 2, 'ChronicCond_Depression': 2,
                   'ChronicCond_Diabetes': 2, 'ChronicCond_IschemicHeart': 2, 'ChronicCond_Osteoporasis': 2,
                   'ChronicCond_rheumatoidarthritis': 2, 'ChronicCond_stroke': 2, 'Gender': 2 },
                  0)
```

In [336…
```
# data1.RenalDiseaseIndicator.unique() #array(['0', 'Y'], dtype=object)
# Replace values with a binary annotation

data1 = data1.replace({'RenalDiseaseIndicator': 'Y'}, 1).astype({'RenalDiseaseIndicator': 'int64'})#array([0, 1], dtype=int64)
```

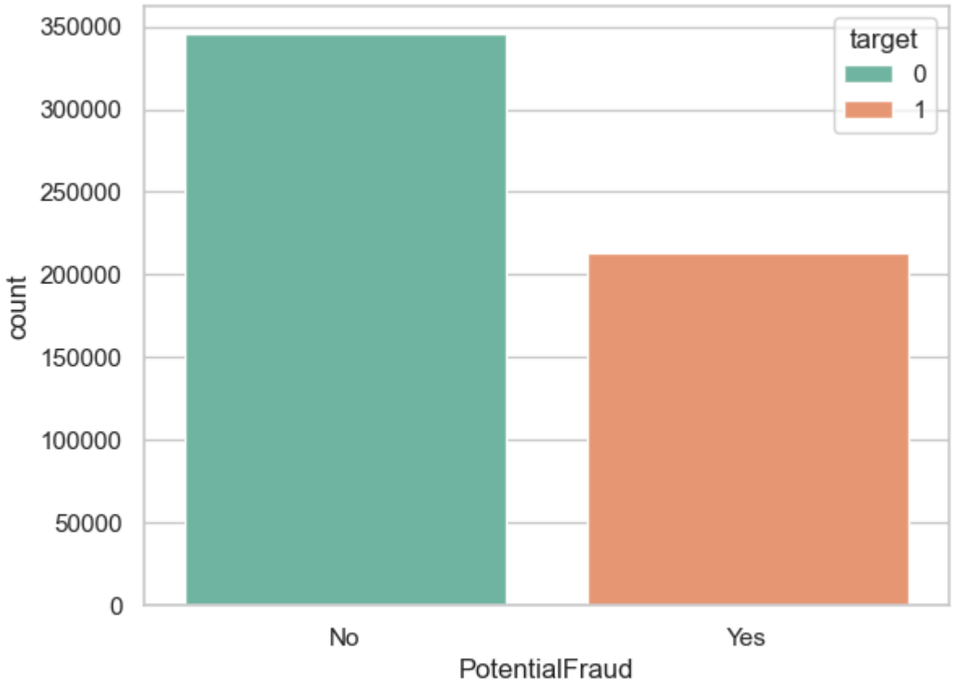
In [337…
```
#merging the dataset created in above step with data
data_final = pd.merge(data1, data , how = 'outer', on = 'Provider' )
```

In [338…
```
# adding extra column target, having binary annotation
data_final['target'] = data_final['PotentialFraud']

# representing potential fraud and 0 representing non potential fraud
data_final['target'] = data_final['target'].map({"Yes":1,"No":0})
data_final['target'] = data_final['target'].astype('category',copy=False)
```

In [339…
```
# checking the distribution of target variable in the final dataset

print(data_final.PotentialFraud.value_counts())
sns.countplot(x='PotentialFraud', data=data_final, palette='Set2', hue='target' )
```

```
PotentialFraud
No     345415
Yes    212796
Name: count, dtype: int64
```

Out[339…
```
<Axes: xlabel='PotentialFraud', ylabel='count'>
```



In [340…

```
# Now we have a final dataset with all the features from inpatient, outpatient and beneficiary datasets,
# along with the target variable indicating potential fraud. We can now proceed to build machine learning models to predict fraudulent providers based on these features.

print(data_final.shape)
data_final.head(5)
```

```
(558211, 58)
```

Out[340…

	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	Provider	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician	OtherPhysician	AdmissionDt	…	ChronicCond_IschemicHeart	C
0	BENE102690	CLM49733	2009-05-04	2009-05-18	PRV51001	42000	PHY383193	PHY378091	PHY389113	2009-05-04	…	1	
1	BENE106078	CLM419490	2009-06-16	2009-06-16	PRV51001	90	PHY345302	NaN	NaN	NaN	…	1	
2	BENE11727	CLM733300	2009-12-17	2009-12-17	PRV51001	20	PHY383007	NaN	PHY383007	NaN	…	1	
3	BENE123416	CLM172987	2009-02-02	2009-02-02	PRV51001	100	PHY345302	NaN	PHY345302	NaN	…	1	
4	BENE126477	CLM361230	2009-05-15	2009-05-15	PRV51001	200	PHY354616	PHY405310	PHY405310	NaN	…	1	

5 rows × 58 columns

In [341…

```
#check columns for NaN values

data_final.isna().sum()
```

Out[341…

```
BeneID                                0
ClaimID                               0
ClaimStartDt                          0
ClaimEndDt                            0
Provider                              0
InscClaimAmtReimbursed                0
AttendingPhysician                 1508
OperatingPhysician               443764
OtherPhysician                   358475
AdmissionDt                      517737
ClmAdmitDiagnosisCode            412312
DeductibleAmtPaid                   899
DischargeDt                      517737
DiagnosisGroupCode               517737
ClmDiagnosisCode_1                10453
ClmDiagnosisCode_2               195606
ClmDiagnosisCode_3               315156
ClmDiagnosisCode_4               393675
ClmDiagnosisCode_5               446287
ClmDiagnosisCode_6               473819
ClmDiagnosisCode_7               492034
ClmDiagnosisCode_8               504767
ClmDiagnosisCode_9               516396
ClmDiagnosisCode_10              553201
ClmProcedureCode_1               534901
ClmProcedureCode_2               552721
ClmProcedureCode_3               557242
ClmProcedureCode_4               558093
ClmProcedureCode_5               558202
ClmProcedureCode_6               558211
is_admitted                           0
Physician_Team                   467602
DOB                                   0
DOD                              554080
Gender                                0
Race                                  0
RenalDiseaseIndicator                 0
State                                 0
County                                0
NoOfMonths_PartACov                   0
NoOfMonths_PartBCov                   0
ChronicCond_Alzheimer                 0
ChronicCond_Heartfailure              0
ChronicCond_KidneyDisease             0
ChronicCond_Cancer                    0
ChronicCond_ObstrPulmonary            0
ChronicCond_Depression                0
ChronicCond_Diabetes                  0
ChronicCond_IschemicHeart             0
ChronicCond_Osteoporasis              0
ChronicCond_rheumatoidarthritis       0
ChronicCond_stroke                    0
IPAnnualReimbursementAmt              0
IPAnnualDeductibleAmt                 0
OPAnnualReimbursementAmt              0
OPAnnualDeductibleAmt                 0
PotentialFraud                        0
target                                0
dtype: int64
```

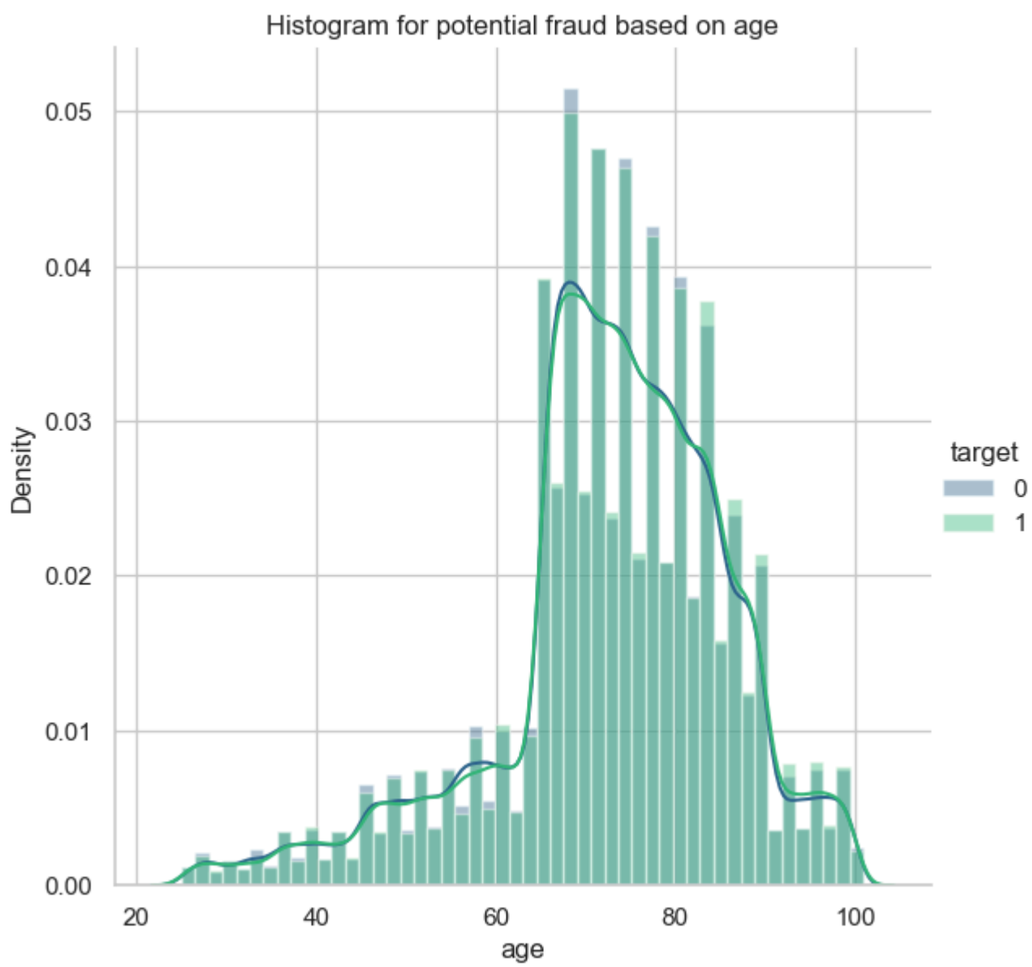
In [342…

```
#Adding a column is_dead  = 1  DOD is given else is_dead = 0

data_final.loc[data_final['DOD'].isnull(), 'Is_Dead'] = '0'
data_final.loc[(data_final['DOD'].notnull()), 'Is_Dead'] = '1'
```

As we have DOD for only few BeneID,so we can only determine the eaxct age of the patinets who has died.we cannot get the age of the patonets who might be alive.So its better that we calculate the age of the patient when we submiited his claim as we have record for ClaimStartDt

In [343…

```
# Now we will analyze the age distribution of beneficiaries associated with fraudulent providers.
# We will calculate the age of beneficiaries at the time of claim and plot the distribution of age for potential fraud and non potential fraud cases.

data_final['DOB'] =  pd.to_datetime(data_final['DOB'], format='%Y-%m-%d')
data_final['ClaimStartDt'] = pd.to_datetime(data_final['ClaimStartDt'], format='%Y-%m-%d')

#now = pd.to_datetime('2020-09-01' , format = '%Y-%m-%d')
data_final['DOB'] = data_final['DOB'].where(data_final['DOB'] < data_final['ClaimStartDt'])
#data_final['age'] = (data_final['ClaimStartDt'] - data_final['DOB']).astype('<m8[Y]')
data_final['age'] = (data_final['ClaimStartDt'] - data_final['DOB']).dt.days // 365


#plotting age
sns.FacetGrid(data_final, hue="target" , height=6, palette='viridis').map(sns.distplot, "age",).add_legend()
plt.title('Histogram for potential fraud based on age')
plt.show()
plt.tight_layout()
```




```
<Figure size 640x480 with 0 Axes>
```

Conclusion

Age

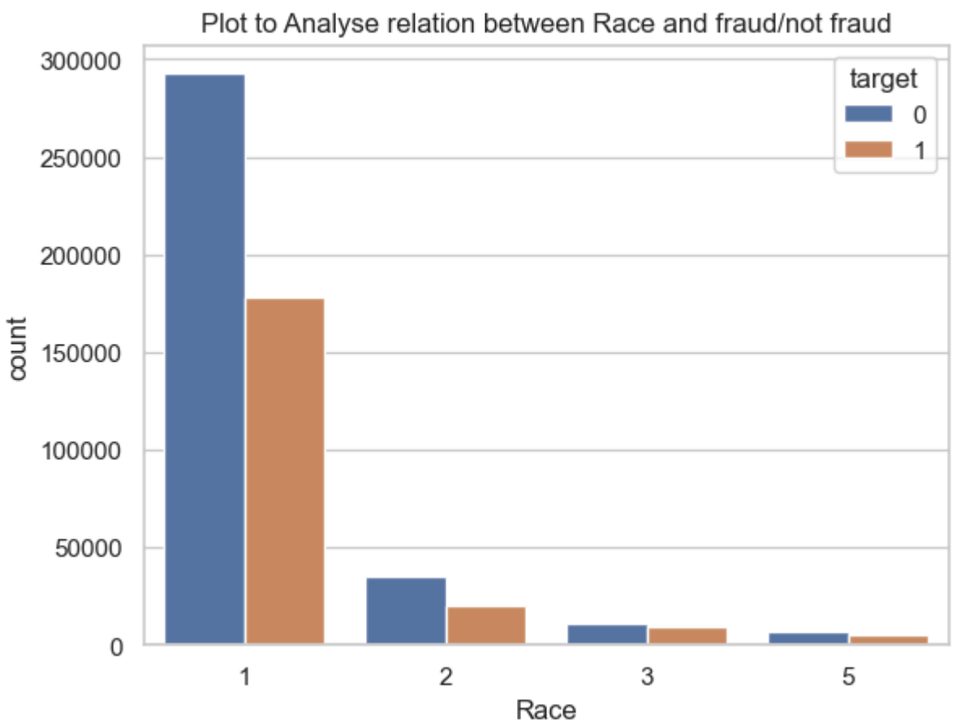
- We don't see any difference in distribution of age that may enable us to flag potential fraud.
- But we see increasing trend in the potential fraud cases for patients age > 65 Also most of the patient who applied for claim fall in this age range only.
- For our analysis we will drop the DOB and DOD.

In [344…

```
# Now we will analyze the relationship between Race and potential fraud cases in the dataset.

ax = sns.countplot(x='Race',data=data_final,hue='target')
ax.set_title("Plot to Analyse relation between Race and fraud/not fraud")
```

Out[344…

```
Text(0.5, 1.0, 'Plot to Analyse relation between Race and fraud/not fraud')
```



Conclusion

It is very important to see that most fraudulent cases is with patients belonging to a particular Race and that is labelled as 1

In [345…

```
# Since we didn't have any use for DOD and DOB columns in our analysis, we will drop these columns from the final dataset.

data_final = data_final.drop(['DOD'], axis = 1)
data_final = data_final.drop(['DOB'], axis = 1)
```

In [346…

```
#calculating number of days a patient was admitted in the hospital and then removing admit and discharge date
#outpatients dont get admitted so we keep 0 for them


data_final['AdmissionDt'] = pd.to_datetime(data_final['AdmissionDt'] , format = '%Y-%m-%d')
data_final['DischargeDt'] = pd.to_datetime(data_final['DischargeDt'],format = '%Y-%m-%d')
data_final['Num_admit_days'] = ((data_final['DischargeDt'] - data_final['AdmissionDt']).dt.days)+1
data_final.loc[data_final['is_admitted'] == 0, 'Num_admit_days'] = '0'

data_final = data_final.drop(['DischargeDt'], axis = 1)
data_final = data_final.drop(['AdmissionDt'], axis = 1)
```

In [347…

```
#check columns for NaN values
data_final.isna().sum()
```

Out[347…

```
BeneID                                0
ClaimID                               0
ClaimStartDt                          0
ClaimEndDt                            0
Provider                              0
InscClaimAmtReimbursed                0
AttendingPhysician                 1508
OperatingPhysician               443764
OtherPhysician                   358475
ClmAdmitDiagnosisCode            412312
DeductibleAmtPaid                   899
DiagnosisGroupCode               517737
ClmDiagnosisCode_1                10453
ClmDiagnosisCode_2               195606
ClmDiagnosisCode_3               315156
ClmDiagnosisCode_4               393675
ClmDiagnosisCode_5               446287
ClmDiagnosisCode_6               473819
ClmDiagnosisCode_7               492034
ClmDiagnosisCode_8               504767
ClmDiagnosisCode_9               516396
ClmDiagnosisCode_10              553201
ClmProcedureCode_1               534901
ClmProcedureCode_2               552721
ClmProcedureCode_3               557242
ClmProcedureCode_4               558093
ClmProcedureCode_5               558202
ClmProcedureCode_6               558211
is_admitted                           0
Physician_Team                   467602
Gender                                0
Race                                  0
RenalDiseaseIndicator                 0
State                                 0
County                                0
NoOfMonths_PartACov                   0
NoOfMonths_PartBCov                   0
ChronicCond_Alzheimer                 0
ChronicCond_Heartfailure              0
ChronicCond_KidneyDisease             0
ChronicCond_Cancer                    0
ChronicCond_ObstrPulmonary            0
ChronicCond_Depression                0
ChronicCond_Diabetes                  0
ChronicCond_IschemicHeart             0
ChronicCond_Osteoporasis              0
ChronicCond_rheumatoidarthritis       0
ChronicCond_stroke                    0
IPAnnualReimbursementAmt              0
IPAnnualDeductibleAmt                 0
OPAnnualReimbursementAmt              0
OPAnnualDeductibleAmt                 0
PotentialFraud                        0
target                                0
Is_Dead                               0
age                                   0
Num_admit_days                        0
dtype: int64
```

In [348…

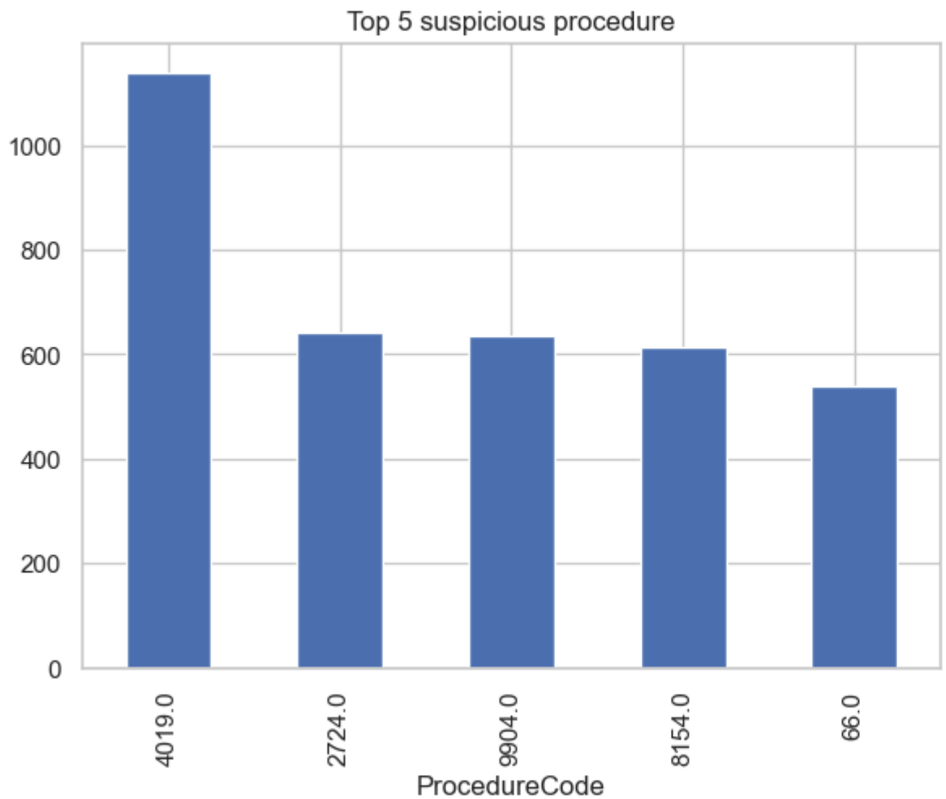
```python
#replacing NaN value with 0
data_final.loc[data_final['DeductibleAmtPaid'].isnull(),'DeductibleAmtPaid'] = 0
```

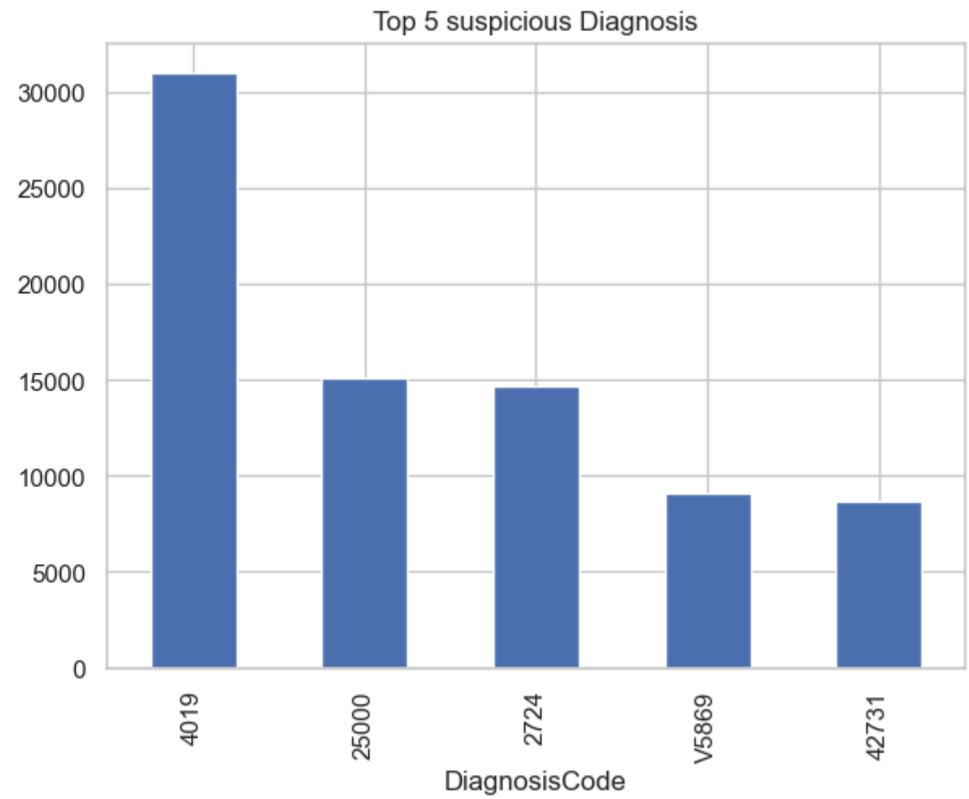
In [349…

```python
# Now we will analyze the top procedure codes and diagnosis codes associated with potential fraud cases in the dataset.
# We will concatenate all the procedure code columns and diagnosis code columns into two separate dataframes
# and then plot the top 5 procedure codes and diagnosis codes associated with potential fraud cases in the dataset.

potential_fraud_df =data_final.loc[data_final['PotentialFraud'] == 'Yes']

df1 = pd.DataFrame(columns = ['ProcedureCode'])
df1['ProcedureCode'] = pd.concat([potential_fraud_df["ClmProcedureCode_1"],
                                  potential_fraud_df["ClmProcedureCode_2"],
                                  potential_fraud_df["ClmProcedureCode_3"],
                                  potential_fraud_df["ClmProcedureCode_4"],
                                  potential_fraud_df["ClmProcedureCode_5"],
                                  potential_fraud_df["ClmProcedureCode_6"]], axis=0, sort=True).dropna()

df2 = pd.DataFrame(columns = ['DiagnosisCode'])
df2['DiagnosisCode'] = pd.concat([potential_fraud_df["ClmDiagnosisCode_1"],
                                  potential_fraud_df["ClmDiagnosisCode_2"],
                                  potential_fraud_df["ClmDiagnosisCode_3"],
                                  potential_fraud_df["ClmDiagnosisCode_4"],
                                  potential_fraud_df["ClmDiagnosisCode_5"],
                                  potential_fraud_df["ClmDiagnosisCode_6"],
                                  potential_fraud_df["ClmDiagnosisCode_7"],
                                  potential_fraud_df["ClmDiagnosisCode_8"],
                                  potential_fraud_df["ClmDiagnosisCode_9"],
                                  potential_fraud_df["ClmDiagnosisCode_10"]], axis=0, sort=True).dropna()

plt.figure(figsize=(15, 5))
plt.subplot(121)
df1.ProcedureCode.value_counts().head(5).plot(kind = 'bar' , title ='Top 5 suspicious procedure')

plt.subplot(122)
df2.DiagnosisCode.value_counts().head(5).plot(kind = 'bar' , title ='Top 5 suspicious Diagnosis')
```

Out[349…

```
<Axes: title={'center': 'Top 5 suspicious Diagnosis'}, xlabel='DiagnosisCode'>
```





In [350…

```
# Defining a list of all diagnosis and procedure code columns to replace NaN values with 0

Diag_proce_col = ['ClmAdmitDiagnosisCode', 'ClmDiagnosisCode_1', 'ClmDiagnosisCode_10',
                  'ClmDiagnosisCode_2', 'ClmDiagnosisCode_3', 'ClmDiagnosisCode_4',
                  'ClmDiagnosisCode_5', 'ClmDiagnosisCode_6', 'ClmDiagnosisCode_7',
                  'ClmDiagnosisCode_8', 'ClmDiagnosisCode_9', 'ClmProcedureCode_1',
                  'ClmProcedureCode_2', 'ClmProcedureCode_3', 'ClmProcedureCode_4',
                  'ClmProcedureCode_5', 'ClmProcedureCode_6']

data_final[Diag_proce_col]= data_final[Diag_proce_col].replace({np.nan:0})
data_final.head(5)
```

Out[350…

	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	Provider	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician	OtherPhysician	ClmAdmitDiagnosisCode	...	ChronicCond_stroke
0	BENE102690	CLM49733	2009-05-04	2009-05-18	PRV51001	42000	PHY383193	PHY378091	PHY389113	41401	...	0
1	BENE106078	CLM419490	2009-06-16	2009-06-16	PRV51001	90	PHY345302	NaN	NaN	0	...	0
2	BENE11727	CLM733300	2009-12-17	2009-12-17	PRV51001	20	PHY383007	NaN	PHY383007	0	...	0
3	BENE123416	CLM172987	2009-02-02	2009-02-02	PRV51001	100	PHY345302	NaN	PHY345302	0	...	0
4	BENE126477	CLM361230	2009-05-15	2009-05-15	PRV51001	200	PHY354616	PHY405310	PHY405310	0	...	0

5 rows × 57 columns

In [351…

```
# Now we will create new features based on the top 5 procedure codes and diagnosis codes associated with potential fraud cases in the dataset.
```

```python
def encoded_cat(dataset, feature_to_encode='',col_list=[]):
    """This function returns top 5 cat column useful in determining potential fraud"""
    outer_list =[]
    for col in col_list:
        list_1 = list()

        for item in list(dataset[col]):
            if str(item) == str(feature_to_encode):
                list_1.append(1)
            else:
                list_1.append(0)

        outer_list.append(list_1)

    li_sum = np.array([0]*558211)

    for i in range(0,len(outer_list)):
        li1 = np.array(outer_list[i])
        li_sum = li_sum + li1

    return li_sum


procedure_col = ['ClmProcedureCode_1','ClmProcedureCode_2','ClmProcedureCode_3','ClmProcedureCode_4','ClmProcedureCode_5','ClmProcedureCode_6']

diagnosis_col = ['ClmDiagnosisCode_1','ClmDiagnosisCode_2','ClmDiagnosisCode_3','ClmDiagnosisCode_4','ClmDiagnosisCode_5','ClmDiagnosisCode_6','ClmDiagnosisCode_7','ClmDiagnos


#top 5 procedure
# 4019.0, 2724.0, 9904.0, 8154.0, 66.0
data_final['pr_4019'] = encoded_cat(data_final,'4019.0',procedure_col)
data_final['pr_2724'] = encoded_cat(data_final,'2724.0',procedure_col)
data_final['pr_9904'] = encoded_cat(data_final,'9904.0',procedure_col)
data_final['pr_8154'] = encoded_cat(data_final,'8154.0',procedure_col)
data_final['pr_66'] =   encoded_cat(data_final,'66.0',procedure_col)

#top 5 diagnosis
#4019,25000,2724,V5869,42731
data_final['di_4019'] = encoded_cat(data_final,'4019',diagnosis_col)
data_final['di_25000'] = encoded_cat(data_final,'25000',diagnosis_col)
data_final['di_2724'] = encoded_cat(data_final,'2724',diagnosis_col)
data_final['di_V5869'] = encoded_cat(data_final,'V5869',diagnosis_col)
data_final['di_42731'] =   encoded_cat(data_final,'42731',diagnosis_col)
```

In [352…

```python
# Now we will replace all the values in the diagnosis and procedure code columns with binary annotations, where 1 indicates the presence of a code and 0 indicates its absence


for i in Diag_proce_col:
    data_final[i][data_final[i]!=0]= 1

data_final[Diag_proce_col].head(5)
```

Out[352…

	ClmAdmitDiagnosisCode	ClmDiagnosisCode_1	ClmDiagnosisCode_10	ClmDiagnosisCode_2	ClmDiagnosisCode_3	ClmDiagnosisCode_4	ClmDiagnosisCode_5	ClmDiagnosisCode_6	ClmDiagnosisCo
0	1	1	0	1	1	1	1	1	
1	0	1	0	0	0	0	0	0	
2	0	1	0	1	0	0	0	0	
3	0	1	0	0	0	0	0	0	
4	0	1	0	1	1	1	1	1	

In [353…

```
# Diagnosis and procedure code columns are currently of object type, we will convert them to float type before replacing values with binary annotations.

data_final[Diag_proce_col]= data_final[Diag_proce_col].astype(float)
```

In [354…

```
#adding column total_num_diag
data_final['total_num_diag'] = 0
for col in diagnosis_col :
    data_final['total_num_diag']  = data_final['total_num_diag'] + data_final[col]
```

In [355…

```
#adding column total_num_proce

data_final['total_num_proce'] = 0
for col in procedure_col :
    data_final['total_num_proce']  = data_final['total_num_proce'] + data_final[col]
```

In [356…

```
# Now we have two new features total_num_diag and total_num_proce which represent
# the total number of diagnosis codes and procedure codes associated with each claim.
# We will convert these columns to float type before replacing values with binary annotations.

data_final['total_num_diag'] =  data_final['total_num_diag'].astype(float)
data_final['total_num_proce'] =  data_final['total_num_proce'].astype(float)
```

Checking if we have duplicate rows

In [357…

```
#Checking we how many distinct BeneID we have in our data
#len(list(tr_data_final.BeneID.value_counts()==1))
len(list(data_final['BeneID'].unique()))
```

Out[357… 138556

In [358…

```
#Checking we how many distinct BeneID we have in our data
len(list(data_final['ClaimID'].unique()))
```

Out[358… 558211

Conclusion

We have data for 138556 BeneID. Means one beneficiary might have applied for claim multiple times Also we have 558211 distinct claimIDs, which means we have separate record for each beneficiary every time he applied for claims No duplicate rows are present in our dataset

In [359…

```
# Now we will analyze the distribution of claims across different beneficiaries in the dataset.
# We will check how many claims are associated with each beneficiary and plot the distribution of claims per beneficiary.
```

```python
val_counts_ = data_final['BeneID'].value_counts()
data_final_beneid_df = val_counts_.to_frame() #store this information in dataframe

data_final_beneid_df.columns = ['count']
data_final_beneid_df['BeneID'] = data_final_beneid_df.index
data_final_beneid_df
```

Out[359…

	count	BeneID
BeneID		
BENE59303	29	BENE59303
BENE42721	29	BENE42721
BENE118316	29	BENE118316
BENE36330	28	BENE36330
BENE44241	28	BENE44241
...	...	...
BENE28565	1	BENE28565
BENE28925	1	BENE28925
BENE29340	1	BENE29340
BENE80992	1	BENE80992
BENE86999	1	BENE86999

138556 rows × 2 columns

In [360…

```python
# Now we will calculate the total amount reimbursed and mean amount reimbursed for each beneficiary in the dataset.

# Use a map for instant lookup
totals_map = data_final.groupby('BeneID')['InscClaimAmtReimbursed'].sum()

total_amt_list = data_final_beneid_df['BeneID'].map(totals_map).tolist()
mean_amt_list = (total_amt_list / data_final_beneid_df['count']).tolist()
```

In [361…

```python
# Now we will add these two new features total_amt and mean_amt to the data_final_beneid_df dataframe which contains the count of claims for each beneficiary.

# 1. Calculate the totals from the MAIN claims data
# This creates a Series where the Index is BeneID and the values are the sums
total_reimbursements = data_final.groupby('BeneID')['InscClaimAmtReimbursed'].sum()

# 2. ADD the column to your beneficiary summary dataframe
# This maps the totals to the correct BeneID rows automatically
data_final_beneid_df['total_amt'] = data_final_beneid_df['BeneID'].map(total_reimbursements)

# 3. Handle Missing Data (Step 5 of your task)
# If a BeneID had no claims, the map results in NaN. We justify filling with 0.
data_final_beneid_df['total_amt'] = data_final_beneid_df['total_amt'].fillna(0)

# 4. Now calculate the mean (Engineering a new feature)
data_final_beneid_df['mean_amt'] = data_final_beneid_df['total_amt'] / data_final_beneid_df['count']
```

```
# 5. Extract to list (if still needed for your specific logic)
mean_list = data_final_beneid_df['mean_amt'].tolist()
```

In [362…

```
# Now we will add these two new features total_amt and mean_amt to the data_final_beneid_df dataframe which contains the count of claims for each beneficiary.

data_final_beneid_df['mean_InscClaimAmtReimbursed']=mean_list
data_final_beneid_df['total_InscClaimAmtReimbursed']= total_amt_list
data_final_beneid_df
```

Out[362…

	count	BeneID	total_amt	mean_amt	mean_InscClaimAmtReimbursed	total_InscClaimAmtReimbursed
BeneID						
BENE59303	29	BENE59303	34100	1175.862069	1175.862069	34100
BENE42721	29	BENE42721	35310	1217.586207	1217.586207	35310
BENE118316	29	BENE118316	42950	1481.034483	1481.034483	42950
BENE36330	28	BENE36330	61090	2181.785714	2181.785714	61090
BENE44241	28	BENE44241	41620	1486.428571	1486.428571	41620
...	...	...	...	...	...	...
BENE28565	1	BENE28565	30	30.000000	30.000000	30
BENE28925	1	BENE28925	400	400.000000	400.000000	400
BENE29340	1	BENE29340	90	90.000000	90.000000	90
BENE80992	1	BENE80992	57000	57000.000000	57000.000000	57000
BENE86999	1	BENE86999	40	40.000000	40.000000	40

138556 rows × 6 columns

In [363…

```
# Now we have a new dataframe data_final_beneid_df which contains the count of claims,
# total amount reimbursed and mean amount reimbursed for each beneficiary in the dataset.

data_final_beneid_df.to_csv('tr_data_final_beneid_df.csv',index= False)
```

In [364…

```
# Now we will merge this new dataframe with the data_final dataframe to add these new features to our final dataset.

data_final = data_final.reset_index(drop=True)
data_final_beneid_df = data_final_beneid_df.reset_index(drop=True)


data_final = pd.merge(data_final, data_final_beneid_df, how='outer', on='BeneID')
print("Dataset shape : ", data_final.shape)
data_final.head(5)
```

```
Dataset shape :  (558211, 74)
```

Out[364…

	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	Provider	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician	OtherPhysician	ClmAdmitDiagnosisCode	…	di_2724	di_V586
0	BENE100000	CLM126832	2009-01-08	2009-01-08	PRV57172	50	PHY383401	NaN	NaN	0.0	…	0	
1	BENE100000	CLM351838	2009-05-10	2009-05-10	PRV57172	70	PHY370909	NaN	PHY363377	0.0	…	0	
2	BENE100001	CLM626521	2009-10-12	2009-10-12	PRV52145	10	PHY430032	NaN	PHY418077	1.0	…	0	
3	BENE100001	CLM633318	2009-10-16	2009-10-16	PRV54683	100	PHY408282	NaN	NaN	1.0	…	0	
4	BENE100001	CLM332544	2009-04-30	2009-05-02	PRV54890	90	PHY343317	PHY408367	PHY408367	1.0	…	0	

5 rows × 74 columns

In [365…

```
data_final.columns
```

Out[365…

```
Index(['BeneID', 'ClaimID', 'ClaimStartDt', 'ClaimEndDt', 'Provider',
       'InscClaimAmtReimbursed', 'AttendingPhysician', 'OperatingPhysician',
       'OtherPhysician', 'ClmAdmitDiagnosisCode', 'DeductibleAmtPaid',
       'DiagnosisGroupCode', 'ClmDiagnosisCode_1', 'ClmDiagnosisCode_2',
       'ClmDiagnosisCode_3', 'ClmDiagnosisCode_4', 'ClmDiagnosisCode_5',
       'ClmDiagnosisCode_6', 'ClmDiagnosisCode_7', 'ClmDiagnosisCode_8',
       'ClmDiagnosisCode_9', 'ClmDiagnosisCode_10', 'ClmProcedureCode_1',
       'ClmProcedureCode_2', 'ClmProcedureCode_3', 'ClmProcedureCode_4',
       'ClmProcedureCode_5', 'ClmProcedureCode_6', 'is_admitted',
       'Physician_Team', 'Gender', 'Race', 'RenalDiseaseIndicator', 'State',
       'County', 'NoOfMonths_PartACov', 'NoOfMonths_PartBCov',
       'ChronicCond_Alzheimer', 'ChronicCond_Heartfailure',
       'ChronicCond_KidneyDisease', 'ChronicCond_Cancer',
       'ChronicCond_ObstrPulmonary', 'ChronicCond_Depression',
       'ChronicCond_Diabetes', 'ChronicCond_IschemicHeart',
       'ChronicCond_Osteoporasis', 'ChronicCond_rheumatoidarthritis',
       'ChronicCond_stroke', 'IPAnnualReimbursementAmt',
       'IPAnnualDeductibleAmt', 'OPAnnualReimbursementAmt',
       'OPAnnualDeductibleAmt', 'PotentialFraud', 'target', 'Is_Dead', 'age',
       'Num_admit_days', 'pr_4019', 'pr_2724', 'pr_9904', 'pr_8154', 'pr_66',
       'di_4019', 'di_25000', 'di_2724', 'di_V5869', 'di_42731',
       'total_num_diag', 'total_num_proce', 'count', 'total_amt', 'mean_amt',
       'mean_InscClaimAmtReimbursed', 'total_InscClaimAmtReimbursed'],
      dtype='object')
```

In [366…

```
# Now we will convert the mean_InscClaimAmtReimbursed and total_InscClaimAmtReimbursed columns to float type before replacing values with binary annotations.

data_final[['mean_InscClaimAmtReimbursed','total_InscClaimAmtReimbursed']]= data_final[['mean_InscClaimAmtReimbursed','total_InscClaimAmtReimbursed']].astype(float)
```

In [367…

```
# Now we will replace the values in mean_InscClaimAmtReimbursed and total_InscClaimAmtReimbursed columns with binary annotations, where 1 indicates that the amount is above a

def N_unique_values(df):
    """This function finds the unique values in a df row"""
    return np.array([len(set([i for i in x[~pd.isnull(x)]])) for x in df.values])
```

In [368…

```
#number of unique physicians for each patient
data_final['N_unique_Physicians'] = N_unique_values(data_final[['AttendingPhysician', 'OperatingPhysician', 'OtherPhysician']])

#encoding types of physicians into numeric values
data_final[['AttendingPhysician', 'OperatingPhysician', 'OtherPhysician']] = np.where(data_final[['AttendingPhysician','OperatingPhysician',
                                                                                                  'OtherPhysician']].isnull(), 0, 1)
```

```
# number of different physicians who attend a patient
data_final['N_Types_Physicians'] = data_final['AttendingPhysician'] +  data_final['OperatingPhysician'] + data_final['OtherPhysician']

# patients who was attended by only 1 physicians
data_final['Same_Physician'] = data_final.apply(lambda x: 1 if (x['N_unique_Physicians'] == 1 and x['N_Types_Physicians'] > 1) else 0,axis=1)

#determine if 1 physician has multi[le role to attend a patient
data_final['Same_Physician2'] = data_final.apply(lambda x: 1 if (x['N_unique_Physicians'] == 2 and x['N_Types_Physicians'] > 2) else 0,axis=1)

data_final[['N_unique_Physicians','N_Types_Physicians','Same_Physician','Same_Physician2']].head()
```

Out[368…

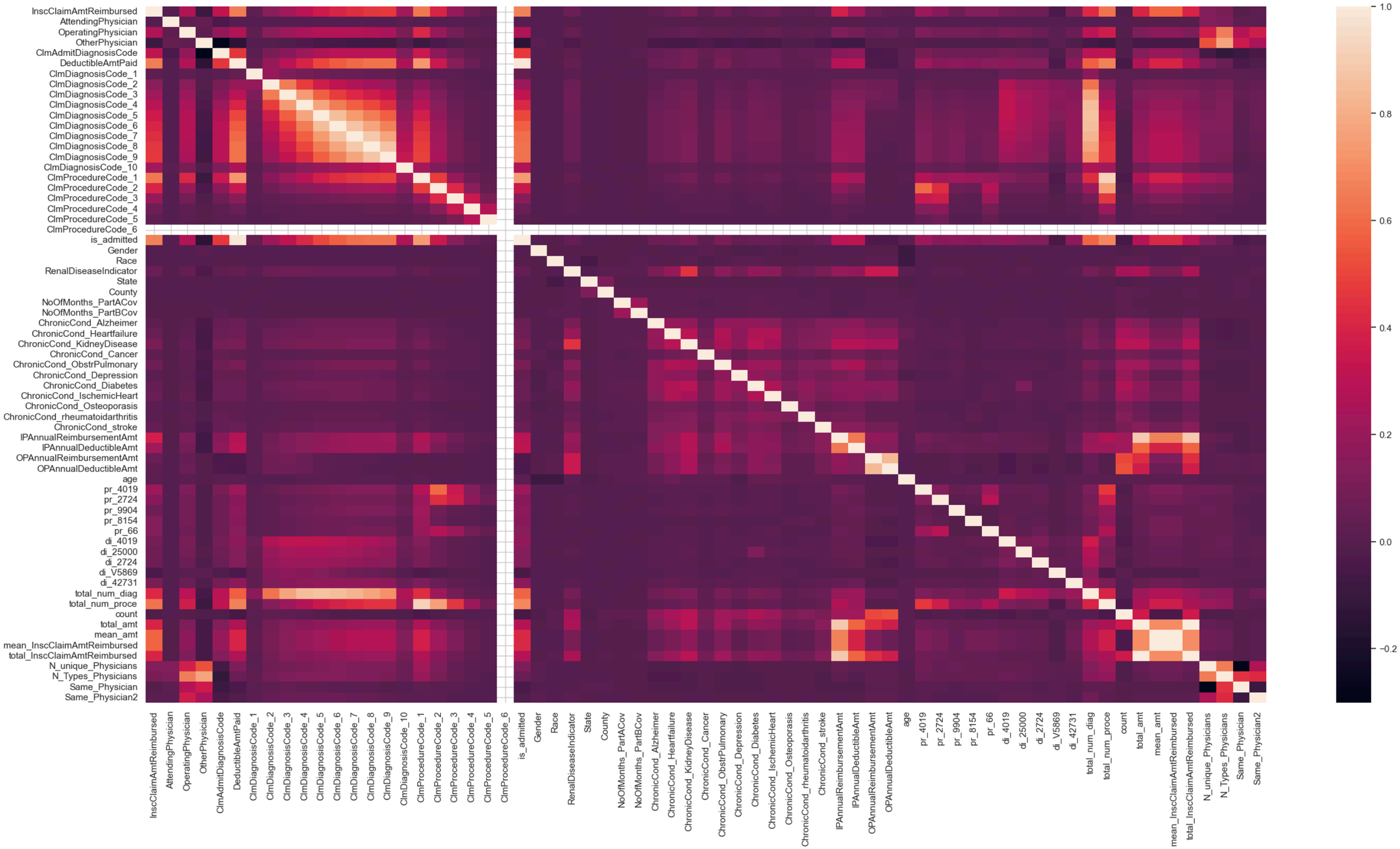
	N_unique_Physicians	N_Types_Physicians	Same_Physician	Same_Physician2
0	1	1	0	0
1	2	2	0	0
2	2	2	0	0
3	1	1	0	0
4	2	3	0	1

In [369…

```
#plot correlation heat map between features
sns.set(context='notebook', style='whitegrid')
plt.figure(figsize=(30, 15))
corr = data_final.corr( numeric_only=True)
sns.heatmap(corr, annot=False)
```

Out[369…

```
<Axes: >
```



In [ ]:
```
#replace NaN val with 0
#data_final.DiagnosisGroupCode
data_final['DiagnosisGroupCode']= data_final['DiagnosisGroupCode'].replace({np.nan:0})
```

In [371…
```
#final check to see if a col val is NaN
col_inf_df=pd.DataFrame()
val = data_final.isnull().sum()
col_inf_df['null_col_count']  = val
```

```
col_inf_df['column'] = val.index
col_inf_df.loc[col_inf_df['null_col_count'] !=0]
```

Out[371…

	null_col_count	column
Physician_Team	467602	Physician_Team

In [372…

```
col_inf_df.null_col_count.value_counts()
```

Out[372…

```
null_col_count
0         77
467602     1
Name: count, dtype: int64
```

In [373…

```
for col in data_final.columns:
    print(col)
```

BeneID
ClaimID
ClaimStartDt
ClaimEndDt
Provider
InscClaimAmtReimbursed
AttendingPhysician
OperatingPhysician
OtherPhysician
ClmAdmitDiagnosisCode
DeductibleAmtPaid
DiagnosisGroupCode
ClmDiagnosisCode_1
ClmDiagnosisCode_2
ClmDiagnosisCode_3
ClmDiagnosisCode_4
ClmDiagnosisCode_5
ClmDiagnosisCode_6
ClmDiagnosisCode_7
ClmDiagnosisCode_8
ClmDiagnosisCode_9
ClmDiagnosisCode_10
ClmProcedureCode_1
ClmProcedureCode_2
ClmProcedureCode_3
ClmProcedureCode_4
ClmProcedureCode_5
ClmProcedureCode_6
is_admitted
Physician_Team
Gender
Race
RenalDiseaseIndicator
State
County
NoOfMonths_PartACov
NoOfMonths_PartBCov
ChronicCond_Alzheimer
ChronicCond_Heartfailure
ChronicCond_KidneyDisease
ChronicCond_Cancer
ChronicCond_ObstrPulmonary
ChronicCond_Depression
ChronicCond_Diabetes
ChronicCond_IschemicHeart
ChronicCond_Osteoporasis
ChronicCond_rheumatoidarthritis
ChronicCond_stroke
IPAnnualReimbursementAmt
IPAnnualDeductibleAmt
OPAnnualReimbursementAmt
OPAnnualDeductibleAmt
PotentialFraud
target
Is_Dead
age
Num_admit_days
pr_4019
pr_2724
pr_9904

```
pr_8154
pr_66
di_4019
di_25000
di_2724
di_V5869
di_42731
total_num_diag
total_num_proce
count
total_amt
mean_amt
mean_InscClaimAmtReimbursed
total_InscClaimAmtReimbursed
N_unique_Physicians
N_Types_Physicians
Same_Physician
Same_Physician2
```

In [374…

```
#saving dataframe to csv file to avoid rework
data_final.to_csv("data_final.csv",index= False)
```

In [375…

```
data_final = pd.read_csv("data_final.csv")

print(data_final.shape)
data_final.head(5)
```

```
(558211, 78)
```

Out[375…

	BeneID	ClaimID	ClaimStartDt	ClaimEndDt	Provider	InscClaimAmtReimbursed	AttendingPhysician	OperatingPhysician	OtherPhysician	ClmAdmitDiagnosisCode	...	total_num_proce
0	BENE100000	CLM126832	2009-01-08	2009-01-08	PRV57172	50	1	0	0	0.0	...	0.0
1	BENE100000	CLM351838	2009-05-10	2009-05-10	PRV57172	70	1	0	1	0.0	...	0.0
2	BENE100001	CLM626521	2009-10-12	2009-10-12	PRV52145	10	1	0	1	1.0	...	0.0
3	BENE100001	CLM633318	2009-10-16	2009-10-16	PRV54683	100	1	0	0	1.0	...	0.0
4	BENE100001	CLM332544	2009-04-30	2009-05-02	PRV54890	90	1	1	1	1.0	...	0.0

5 rows × 78 columns

In [ ]:

```
# Now we will drop the columns that are not useful for building machine learning models to predict fraudulent providers.
# We will keep only the relevant features that we have engineered and the target variable in our final dataset.

col_to_remove = ['Provider','BeneID', 'ClaimID', 'ClaimStartDt','ClaimEndDt','AttendingPhysician',\
                 'OperatingPhysician', 'OtherPhysician','ClmAdmitDiagnosisCode','NoOfMonths_PartACov',\
                 'NoOfMonths_PartBCov','DiagnosisGroupCode','PotentialFraud']

data_final.drop(columns=col_to_remove, axis=1, inplace=True)
data_final['target']=data_final['target'].astype(int)
```

In [377…

```
data_final.head()
```

Out[377…

	InscClaimAmtReimbursed	DeductibleAmtPaid	ClmDiagnosisCode_1	ClmDiagnosisCode_2	ClmDiagnosisCode_3	ClmDiagnosisCode_4	ClmDiagnosisCode_5	ClmDiagnosisCode_6	ClmDiagnosisCod
0	50	0.0	1.0	0.0	0.0	0.0	0.0	0.0	
1	70	0.0	1.0	0.0	0.0	0.0	0.0	0.0	
2	10	0.0	1.0	1.0	0.0	0.0	0.0	0.0	
3	100	0.0	1.0	1.0	1.0	0.0	0.0	0.0	
4	90	0.0	1.0	1.0	1.0	1.0	1.0	1.0	

5 rows × 65 columns

Conclusion

To start with we performed some EDA tasks which were remaining from the Milestone 1

We performed the data cleaning and preprocessing.

We performed tasks like below

- Feature Transformation in Calculation of Age, Binary Annotation to medical condition.
- Engineer New feature like Total amount for each Beneficiary and Consolidating Proc and diag codes
- Drop Feature like DOB and DOD, when the feature was not useful for the model
- Performing data extraction like Potential Fraud Provider in the dataset.
- Making Deductible amount as 0 when data is NULL.
- We used the Correlation Heat Map to drop redundant features.